```
<!-- tempconvert1.jsp
     A document that collects a Celsius temperature from a
     client and uses a scriptlet to convert it to Fahrenheit
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Temperature converter </title>
  </head>
  <body>
    <p>
      <%

// Get the Celsius temperature from the form
         String strCtemp = request.getParameter("ctemp");
         float ftemp;

// If this is not the first request (there was a form value),
// convert the value to Fahrenheit
         if (strCtemp != null) {
           ftemp = 1.8f * Integer.parseInt(strCtemp) + 32.0f;
       %>

<!-- Use an expression to display the value of the Fahrenheit
     temperature -->
     Fahrenheit temperature:

       <%= ftemp %>

<!-- Code for the end of the then clause compound
     statement -->
       <%
         }  //** end of if (strCtemp != ...
         else {
       %>

<!-- This is the first request, so display the form
     to collect the Celsius temperature -->
       <form action = "tempconvert1.jsp" method = "get" >
         Celsius temperature:
         <input type = "text" name = "ctemp" />
         <input type = "submit" />
       </form>
     </p>
```

```
<!-- Code for the end of the else clause compound
     statement -->
     <%
         } //** end of else clause
     %>
  </body>
</html>
```

In the first version of JSP, 1.1, all dynamic parts of documents were specified with scriptlets. Of course, embedding a significant amount of Java code in a JSP document defeated the goals of the JSP approach to writing servlets. Putting a large amount of Java code in an XHTML document is not better than putting a large amount of XHTML markup in a servlet. Even the simple JSP document, `tempconvert1.jsp`, illustrates the confusion that results from mixing Java and XHTML in a document.

Recent versions of JSP, which include the JSP Standard Tag Library (JSTL) and the JSP Expression Language, have made it unnecessary to include raw Java code in a JSP document. Furthermore, the use of scriptlets is now being discouraged. For these reasons, this chapter does not further discuss scriptlets.

The focus of the remainder of this chapter is JSP using JSTL.

10.5.4 Expression Language

The use of JSTL requires knowledge of its two primary technologies, the actual tag set of JSTL and the JSP Expression Language.

The JSP Expression Language (EL) is similar to the expressions (but only the expressions) of a scripting language such as JavaScript, at least in terms of simplicity. This similarity is most evident in the type coercion rules, which obviate most of the strict typing and explicit type conversions that are required in writing expressions in strongly typed programming languages such as Java. For example, if a string is added to a number in EL, an attempt will be made to coerce the string to a number. This makes it convenient for dealing with form data, which is always in text form but often represents data of other types. It also makes EL easier for Web designers, who often are not Java programmers. Also, EL has no control statements such as selection and loop control. These are provided by action elements from the JSTL. EL is true to its name—it is just a language for simple expressions.

Syntactically, an EL expression is always introduced with a dollar sign (`$`) and delimited by braces, as shown in the following:

```
${ expression }
```

An EL expression consists of literals, the usual arithmetic operators, implicit variables that allow access to form data, and normal variables. The literals can be numeric, either in the form of floating-point or integer values,

Booleans (`true` or `false`), or strings delimited by either single or double quotes. A variable that has not been assigned a value has the value `null`. The only variables we will use are those created by the JSTL action elements.

The reserved words of EL are as follows:

```
and    div    empty  eq     false  ge     gt     instanceof
le     lt     mod    ne     not    null   or     true
```

Some of these are synonyms for symbolic operators, for example, `le` for <= and `lt` for <. This avoids any problems with having <, which begins tags, in a document.

An EL expression can appear in two places in a JSP document, in template text or in the values of certain attributes of certain action elements. EL often is used to set the attribute values of action elements. Because attributes take string values, the result of the evaluation of an EL expression is always coerced to a string.

EL uses data that comes from several different sources. The most interesting of these for our discussion is the form data sent in a request form, which is made available through the implicit variable, `param`. The `param` variable stores a collection of all of the form data values. To access a particular form data value, the name of the form element is used like a property name in JavaScript, catenated on the collection name with a period. For example, if there is a form element named `address`, it can be accessed with the following:

```
${param.address}
```

If the form element name includes special characters, an alternative access form is used, which is to treat the element name, specified as a literal string, as a subscript into the `param` array, as in the following:

```
${param['cust-address']}
```

EL defines a number of other implicit variables. Most of them are collections of values related to the request header, form values, cookies, and various scope variables. For example, the `pageContext` implicit variable is a reference to an object of class `javax.servlet.http.HttpServletRequest`, which has a long list of information about the request. Among these are `contentType`, `method`, which is the request method (`GET` or `POST`), `remoteAddr`, the IP of the client, and `contentLength`.

Although the values of EL expressions are usually implicitly placed in the result document, it is good to explicitly request such placement. This is accomplished by assigning the expression value to the value attribute of the `out` action element defined in the JSTL Core library. The recommended prefix for this library is `c`. To output the value of the address form element, the following could be used:

```
<c:out value = "${param.address}" />
```

The following example consists of an XHTML document with a form that solicits a temperature in Celsius from the user and a JSP document that pro-

cesses the form, which in this case computes the equivalent temperature in Fahrenheit and returns an HTML document to the user with that value. The XHTML document is `tempconvert2.html`.

```
<!xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- tempconvert2.html
     Get a temperature in Celsius and call a JSTL JSP
     document (tempconvert2.jsp) to convert it to Fahrenheit
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Get a Celsius temperature </title>
  </head>
  <body>
    <form action = "tempconvert2.jsp" method = "post" >
      <p>
        Celsius temperature:
        <input type = "text" name = "ctemp" /> <br />
        <input type = "submit" value = "Convert to Fahrenheit" />
      </p>
    </form>
  </body>
</html>
```

The JSP document to process the form data in `tempconvert2.html` is `tempconvert2.jsp`.

```
<!-- tempconvert2.jsp
     Convert a given temperature in Celsius to Fahrenheit.
     Called by tempconvert2.html
     -->
<%@ page contentType = "text/html" %>
<%@ taglib prefix = "c"
           uri = "http://java.sun.com/jsp/jstl/core" %>
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Temperature converter </title>
  </head>
  <body>
    <p>
      Given temperature in Celsius:
```

```
      <c:out value = "${param.ctemp}" />
      <br /> <br />
      Temperature in Fahrenheit:
      <c:out value = "${(1.8 * param.ctemp) + 32}" />
    </p>
  </body>
</html>
```

Note that although the JSP document, `tempconvert.jsp`, has the form of an XHTML 1.1 document, it cannot be validated as one because of the embedded JSP.

10.5.5 JSTL Control Action Elements

The Core library of JSTL includes a collection of action elements for flow control in a JSP document. The most commonly used of these are `if`, `forEach`, `when`, `choose`, and `otherwise`. The form of an `if` element that has a body, which is the most useful form, is as follows:

```
<c:if test = "boolean expression">
```

JSP elements and/or XHTML markup

```
</c:if>
```

An `if` element could be used to write a JSP document that served as both the requesting document and the responding document. It could determine whether the document was being processed (after being interacted with and sent to the server) by checking whether the `method` implicit variable had been set to `"POST"`. For example:

```
<c:if test = "pageContext.request.method == 'POST'}">
```

JSP elements and/or XHTML markup

```
</c:if>
```

The following is a JSP document for the temperature conversion previously done with `tempconvert2.html` and `tempconvert2.jsp`.

```
<!-- tempconvert3.jsp
     Convert a given temperature in Celsius to Fahrenheit
     This is both the request and the response document
     -->
<%@ page contentType = "text/html" %>
<%@ taglib prefix = "c"
           uri = "http://java.sun.com/jsp/jstl/core" %>
```

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Temperature converter </title>
  </head>
  <body>

      <c:if test = "${pageContext.request.method != 'POST'}">
        <form action = "tempconvert3.jsp" method = "post" >
            Celsius temperature:
            <input type = "text" name = "ctemp" /> <br />
            <input type = "submit"
                   value = "Convert to Fahrenheit" />
        </form>
      </c:if>

      <c:if test = "${pageContext.request.method == 'POST'}">
        Given temperature in Celsius:
             <c:out value = "${param.ctemp}" />
        <br /> <br />
        Temperature in Fahrenheit:
             <c:out value = "${(1.8 * param.ctemp) + 32}" />
      </c:if>

  </body>
</html>
```

Through "view source," one can see the two versions of `tempconvert3.jsp` that come to the browser. These are shown in Figures 10.9 and 10.10.

Figure 10.9 A listing of the initial version of `tempconvert3.jsp`

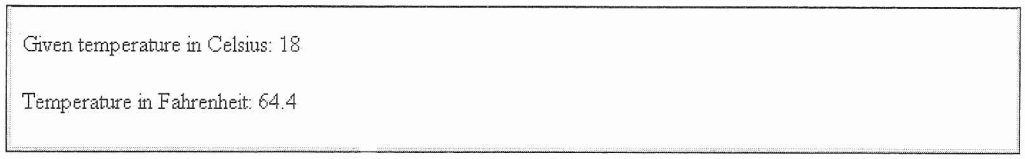


Figure 10.10 The listing of `tempconvert3.jsp` after submitting the original

Checkboxes and menus have multiple values. The `param` implicit variable cannot be used to determine which values are set in the document that handles forms with these components. For this, there is the `paramValues` implicit variable, which has an array of values for each form element. The `forEach` JSTL action element can be used to iterate through the elements of a `paramValues` array.

`forEach` is related to the Perl `foreach` statement—it iterates based on the elements of a collection, an iterator, an enumeration, or an array. The `items` attribute is assigned the data structure on which the iteration is based. The `var` attribute is assigned variable name to which the structure's elements are assigned. For example, consider the following checkboxes:

```
<form method = "post">
  <label>
    <input type = "checkbox" name = "topping"
           value = "extracheese"
           checked = "checked" />     Extra cheese <br />
  </label>
  <label>
    <input type = "checkbox" name = "topping"
           value = "pepperoni" /> Pepperoni <br />
  </label>
  <label>
    <input type = "checkbox" name = "topping"
           value = "olives" /> Olives <br />
  </label>
  <label>
    <input type = "checkbox" name = "topping"
           value = "onions" /> Onions <br />
  </label>
  <label>
    <input type = "checkbox" name = "topping"
           value = "bacon" /> Bacon <br />
  </label>
  <input type = "submit"  value = "Submit" /> <br />
</form>
```

To list the checkboxes that were checked, the following could be used:

```
Pizza Toppings:
<c:forEach items = "${paramValues.topping}"
           var = "top">
  <c:out value = "${top}"> <br />
</c:forEach>
```

The `forEach` element can also be used to control a loop body based on a counter. For this, it uses the `begin`, `end`, and `step` attributes. For example, the following could be used to simply repeat the enclosed code ten times:

```
<c:forEach begin = "1" end = "10">
  ...
<c:/forEach>
```

Radio buttons must be handled differently than checkboxes. All radio buttons in a group have the same name. For this situation, JSTL has three action elements that allow the specification of a form of a switch construct. These three are `choose`, `when`, and `otherwise`. The `choose` element, which takes no attributes, encloses the whole construct. A `when` element specifies one of the selectable sequences of code. The `when` attribute, `test`, is set to an EL expression that describes the Boolean expression that controls entry into the body of the element. The `otherwise` element, which takes no attributes, specifies the code for the case when none of the Boolean expressions in the `when` elements is true. The first `when` element with a true `test` attribute is chosen, so if the `test` attributes of more than one of the `when` elements are true, only one is chosen. Consider the following example, which only displays the radio button that is currently pressed:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- testradio.jsp
     Display radio buttons and use JSP to display which is
     pressed when the form is submitted
     -->
<%@ page contentType = "text/html" %>
<%@ taglib prefix = "c"
           uri = "http://java.sun.com/jsp/jstl/core" %>

<html xmlns = "http://www.w3.org/1999/xhtml">
<head> <title> Test Radio buttons </title>
</head>
<body>
  <form method = "post">
    <p>
      <label>
        <input type = "radio" name = "payment"
               value = "visa" checked = "checked" />
          Visa
      </label>
      <label>
        <input type = "radio" name = "payment"
               value = "mc" />
          Master Charge
      </label>
```

```
      <label>
        <input type = "radio" name = "payment"
               value = "discover" />
          Discover
      </label>
      <label>
        <input type = "radio" name = "payment"
               value = "check" /> Check <br/>
      </label>
      <input type = "submit" value = "Submit" />
    </p>
  </form>

<!-- If the form has been submitted, display the payment method -->
  <c:if test = "${pageContext.request.method == 'POST'}">
    You have chosen the following payment method:
    <c:choose>
      <c:when test = "${param.payment == 'visa'}">
        Visa
      </c:when>
      <c:when test = "${param.payment == 'mc'}">
        Master Charge
      </c:when>
      <c:when test = "${param.payment == 'discover'}">
        Discover
      </c:when>
      <c:otherwise>
        Check
      </c:otherwise>
    </c:choose>
  </c:if>
</body>
</html>
```

Summary

A servlet is a Java program that resides on the Web server and is enacted when requests are received from Web clients. A program called a servlet container, which runs in the Web server, controls the execution of servlets. The most common uses of servlets are as server-side scripts and as alternatives to CGI programs.

Most user-written servlets are extensions to the predefined abstract class `HttpServlet`, which is a predefined descendant of `GenericServlet`, which implements the `Servlet` interface. Any class that is derived from `HttpServ-`

let must override at least one of its methods—most often doGet, doPost, or both. The doGet and doPost methods take two parameters: one to get the input from the client and one to return results to the client. The setContentType method sets the MIME type for the return information. The println method of a PrintWriter object is used to create the return information. The getParameter method is used to get the form values from the inquiry string of a form submission from the client. It is called through the request object parameter.

A Web server can store information about clients on the clients themselves in two ways: by using cookies and by using session tracking. A session begins with the first client request to a Web server and ends when the client's browser is stopped. Cookies are implemented on the server as objects of the Cookie class, which defines a collection of methods for dealing with cookie objects. Each cookie stores a single name/value pair. The server may send a cookie to the client along with the response to the client's request. Each subsequent request made by that client to that server includes the cookies (that are still alive) that have been sent by the server during any prior session. Each cookie has a lifetime, which is assigned with the setMaxAge method of the Cookie class. Cookies are destroyed when their lifetimes end. The servlet attaches a cookie to its response to a client with the addCookie method of the response object. Cookies are obtained from a client request with the getCookies method of the request object.

Session tracking is an alternative to cookies for storing information about a client during a session. In this case, the information is stored in an object of the HTTPSession class, often called a session object, on the server. Such an object can store an unlimited amount of information, whereas a cookie can store just one name/value pair. Session objects live only as long as the session lasts (that is, only as long as the session in which they are created). Values are inserted into a session object with the putValue method and are removed with the removeValue method. The getValue method takes a string as a parameter, which it uses as the name of a session object value. getValue returns the value associated with the given name.

JSP is a collection of several approaches to support dynamic documents on the server. It is an alternative to servlets, putting some form of code in markup, rather than the servlet approach to producing markup with Java code. JSTL provides a set of action elements that form a programming language that has the form of markup. EL is a simple expression language to be used with JSP. The if JSTL element provides a selection construct; the forEach element provides a loop construct; and choose, when, and otherwise provide a multiple selection construct.

Servlets should be used when there is little static content in the return document; JSP should be used when there is little dynamic content.

Review Questions

10.1 What potential advantages do servlets have over CGI programs?

10.2 What is a servlet container?

10.3 Describe the purpose of the life-cycle servlet methods.

10.4 Most user-written servlets extend what predefined class?

10.5 What are the purposes of the `doGet`, `doPost`, and `doPut` methods of the `HttpServlet` class?

10.6 Describe the two parameters to `doGet` and `doPost`.

10.7 What must the first markup output of a servlet to a client be?

10.8 What class of object is used to create XHTML output of a servlet to a client?

10.9 How does a servlet read form data values sent by a client to a servlet?

10.10 What is a session?

10.11 Why would a Web server need to store information on a client about the client's previous requests?

10.12 What is a cookie?

10.13 What do the methods `setMaxAge`, `setValue`, and `getComment` do?

10.14 How is a cookie added to a response by a servlet?

10.15 How does a servlet get a cookie coming from a client?

10.16 What is session tracking?

10.17 How is a name/value pair added to a session object?

10.18 How does a session object get the value of a name?

10.19 How can a servlet determine whether a session object exists for a client request?

10.20 Why should the use of scriplets be restricted?

10.21 What are the two kinds of people who develop and maintain dynamic documents?

10.22 What happens during the compilation process for JSP documents?

10.23 What is template text?

10.24 What are the five parts of the JSTL?

10.25 What is the purpose of the `taglib` directive?

10.26 What is the syntactic form of an EL expression?

10.27 What are the two ways the `param` implicit variable can be used to access form values?

10.28 Describe the syntax and semantics of the `forEach` element when it is used to iterate through a collection.

10.29 Describe the semantics of a `choose` element that includes several `when` elements.

Exercises

10.1 Write a servlet that uses `doGet` to return an XHTML document that provides your name, e-mail address, and mailing address, along with a brief autobiography. Test your servlet with a simple XHTML document.

10.2 Write a servlet that returns a randomly chosen greeting from a list of five different greetings. The greetings must be stored as constant strings in the program.

10.3 Revise the survey sample servlet, `Survey.java`, to display the results of the survey in a table, with female responses in one column and male responses in another.

10.4 Revise the survey sample servlet, `Survey.java`, to record the number of votes so far in the data file and then display that count every time a vote is submitted or a survey result is requested. Also, change the output table so that its data is a percentage of the total votes for the particular gender category.

10.5 Write the XHTML to create a form that collects favorite popular songs, including the name of the song, the composer, and the performing artist or group. This document must call a servlet when the form is submitted and another servlet to request a current list of survey results.

10.6 Modify the servlet for Exercise 10.5 to count the number of visitors and then display that number for each visitor.

10.7 Modify the XHTML form for the election and the servlet `VoteCounter` to allow voters to vote for one additional office. The new office is named Catcatcher. Candidates for Catcatcher are Kitty Catland, Al El Gato, Kitten Katnip, Tommie Cat, and Fred Feline. The election results must be in terms of percentage of the total vote for an office. Votes are not counted if the client did not vote for both offices.

10.8 Rewrite the servlets for Exercise 10.7 to use session tracking rather than cookies.

10.9 Write the XHTML to create a form with the following capabilities:

a. A text widget to collect the user's name

b. Four checkboxes, one each for the following items:
   i. Four 100-watt light bulbs for $2.39
   ii. Eight 100-watt light bulbs for $4.29
   iii. Four 100-watt long-life light bulbs for $3.95
   iv. Eight 100-watt long-life light bulbs for $7.49

c. A collection of three radio buttons that are labeled as follows:
   i. Visa
   ii. MasterCard
   iii. Discover

10.10 Write a servlet that computes the total cost of the ordered light bulbs from Exercise 10.9 after adding 6.2 percent sales tax. The servlet must inform the buyer of exactly what was ordered, in a table.

10.11 Write the XHTML to provide a form that collects names and telephone numbers. The phone numbers must be in the format ddd-ddd-dddd. Write a servlet that checks the submitted telephone number to be sure that it conforms to the required format and then returns a response that indicates whether the number was correct.

10.12 Revise the survey example so that it displays the result as a horizontal bar, similar to a progress bar, ranging from 0–100.

10.13 Emulate the built-in session-tracking mechanism using cookies. *Hint*: Use one cookie for the session ID and a class variable of the servlet to store a table of session objects.

10.14 Write and test a JSP document that displays the form of Exercise 10.9 and produces the same response document as Exercise 10.10.

10.15 Write an XHTML document that displays a form that collects three numbers from the client and calls a JSP document that computes the value of multiplying the three numbers together. The JSP document must use scriptlets.

10.16 Write a single JSP document that does exactly what is prescribed in Exercise 10.15.

10.17 Write a single JSP document that displays a form that collects the radius of a circle. The document must include scriptlets to compute the circumference and area of the circle and display them.

CHAPTER

11

Introduction to PHP

11.1 Origins and Uses of PHP
11.2 Overview of PHP
11.3 General Syntactic Characteristics
11.4 Primitives, Operations, and Expressions
11.5 Output
11.6 Control Statements
11.7 Arrays
11.8 Functions
11.9 Pattern Matching
11.10 Form Handling
11.11 Files
11.12 Cookies
11.13 Session Tracking
Summary • Review Questions • Exercises

This chapter introduces PHP and discusses its use as a server-side scripting language. It begins with a brief look at the origins of PHP, followed by an overview of its primary characteristics and some of its general syntactic conventions. Next, the core language is introduced. Because PHP is close to both Perl and JavaScript, the discussion of its expressions and statements is brief. PHP's arrays, which are different from those of any other language, are then introduced, followed by a description of PHP's functions and their parameter-passing mechanisms. Because PHP uses the same regular expressions for pattern

matching as Perl and JavaScript,[1] regular expressions are not described in this chapter. The form-handling techniques of PHP are discussed next, including a complete example. Finally, both cookies and session tracking in PHP are introduced.

Significant parts of PHP are not covered in this chapter. Among these are references and support for object-oriented programming. PHP access to databases is discussed in Chapter 13, "Database Access through the Web."

11.1 Origins and Uses of PHP

PHP was developed by Rasmus Lerdorf, a member of the Apache Group,[2] in 1994. Its initial purpose was to provide a tool to help Lerdorf track visitors to his personal Web site. In 1995 he developed a package called Personal Home Page Tools, which became the first publicly distributed version of PHP. Originally, PHP was an acronym for Personal Home Page. Later, its user community began using the recursive name PHP: Hypertext Preprocessor, which subsequently forced the original name into obscurity.

Within two years of its release, PHP was being used at a large number of Web sites. By then, the job of managing its development had grown beyond the abilities of a single person, and that task was transferred to a small group of devoted volunteers. PHP is now developed, distributed, and supported as an open-source product. A PHP processor is now resident on most Web servers.

As a server-side scripting language, PHP is naturally used for form handling and database access. Database access has been a prime focus of PHP development; as a result, it has driver support for 15 different database systems. PHP supports the common electronic mail protocols POP3 and IMAP. It also supports the distributed object architectures COM and CORBA.

11.2 Overview of PHP

PHP is a server-side, XHTML-embedded scripting language. As such, it is an alternative to CGI, Microsoft's Active Server Pages (ASP and ASP.NET), Sun's Java Server Pages (JSP), and Allaire's ColdFusion.

In the sense of the way its scripts are interpreted, PHP is related to client-side JavaScript. When a browser finds JavaScript code embedded in an XHTML document it is displaying, it calls the JavaScript interpreter to interpret the script. When a browser requests an XHTML document that includes PHP script, the Web server that provides the document calls the PHP processor. The server determines that a document includes PHP script by the filename extension. If it is `.php`, `.php3`, or `.phtml`, it has embedded PHP.

---

1. Actually, PHP can use two different kinds of regular expressions, POSIX and Perl style.
2. The Apache Group develops and distributes the Apache Web server, among other things.

The PHP processor has two modes of operation, copy mode and interpret mode. It takes a PHP document file as input and produces an XHTML document file. When the PHP processor finds XHTML code (which may include embedded client-side script) in the input file, it simply copies it to the output file. When it encounters PHP script in the input file, it interprets it and sends any output of the script to the output file. This implies that the output from a PHP script must be XHTML or embedded client-side script. This new file (the output file) is sent to the requesting browser. The client never sees the PHP script. If the user clicks *View Source* while the browser is displaying the document, only the XHTML (and embedded client-side script) will be shown, because that is all that ever arrives at the client.

PHP is usually purely interpreted, as is the case with JavaScript. However, recent PHP implementations perform some precompilation, at least on complex scripts, which increases the speed of interpretation.

The syntax and semantics of PHP are closely related to the syntax and semantics of two languages already described in this book, JavaScript and Perl. This should make it relatively easy to learn, assuming the reader has learned one or both of these languages.

PHP uses dynamic typing, as does JavaScript. Variables are not type declared, and they have no intrinsic type. The type of a variable is set every time it is assigned a value, taking on the type of that value. Similar to JavaScript, PHP is far more forgiving than most common programming languages. Dynamic typing is largely responsible for this, but the dynamic nature of its strings and arrays also contributes. PHP's arrays are a merge of the arrays of common programming languages and associative arrays, having the characteristics of both. There is a large collection of functions for creating and manipulating PHP's arrays. PHP supports both procedural and object-oriented programming.

PHP has an extensive library of functions, making it a flexible and powerful tool for server-side software development. Many of the predefined functions are used to provide interfaces to other software systems such as mail and database systems.

As is the case with JavaScript and Perl, language processors for PHP are free and easily obtainable. In addition, the PHP processor is an open-source system. It is available on all common computing platforms. The Web site for official information on PHP is `http://www.php.net`.

11.3 General Syntactic Characteristics

PHP scripts are either embedded in XHTML documents or are in files that are referenced by XHTML documents. PHP code is embedded in XHTML documents by enclosing it between the `<?php` and `?>` tags.

If a PHP script is stored in a different file, it can be brought into a document with the `include` construct, which takes the filename as its parameter. For example:

```
include("table2.inc");
```

This construct causes the contents of the file `table2.inc` to be copied into the document where the call appears. The included file can contain XHTML markup or client-side script, as well as PHP code, but any PHP script it includes must be the content of a `<?php` tag, even if the `include` appears in the content of a `<?php` tag. The PHP interpreter changes from interpret to copy mode when an `include` is encountered.

All variable names in PHP begin with dollar signs (`$`). The part of the name after the dollar sign is like the names of variables in many common programming languages: a letter or an underscore followed by any number (including zero) of letters, digits, or underscores. PHP variable names are case sensitive.

Table 11.1 lists the PHP reserved words. Although variable names in PHP are case sensitive, neither reserved words nor function names are. For example, there is no difference between `while`, `WHILE`, `While`, and `wHiLe`.

Table 11.1 The reserved words of PHP

and	else	global	require	virtual
break	elseif	if	return	xor
case	extends	include	static	while
class	false	list	switch	
continue	for	new	this	
default	foreach	not	true	
do	function	or	var	

PHP allows comments to be specified in three different ways. Single-line comments can be specified either with `#`, as in Perl, or with `//`, as in JavaScript. Multiple-line comments are delimited with `/*` and `*/`, as in many other programming languages. Perl-style comments are most frequently used in PHP scripts.

PHP statements are terminated with semicolons. Braces are used to form compound statements for control structures. Unless used as the body of a function definition, compound statements cannot be blocks (they cannot define locally scoped variables).

11.4 Primitives, Operations, and Expressions

PHP has four scalar types, Boolean, integer, double, and string; two compound types, array and object; and two special types, resource and NULL. In this section, only the scalar types and NULL are discussed. Arrays are discussed in Section 11.7; objects and resource types are not covered in this book.

11.4.1 Variables

Because PHP is dynamically typed, it has no type declarations. In fact, there is no way or need to ever declare the type of a variable.[3] The type of a variable is set every time it is assigned a value. An unassigned variable, sometimes called an *unbound variable*, has the value `NULL`, which is the only value of the NULL type. If an unbound variable is used in an expression, `NULL` is coerced to a value that is dictated by the context of the use. If the context specifies a number, `NULL` is coerced to `0`; if the context specifies a string, `NULL` is coerced to the empty string.

A variable can be tested to determine whether it currently has a value with the `IsSet` function, which takes the variable's name as its parameter and returns a Boolean value. For example, `IsSet($fruit)` returns `TRUE` if `$fruit` currently has a non-`NULL` value, `FALSE` otherwise. A variable that has been assigned a value retains that value until either it is assigned a new value or it is set back to the unassigned state, which is done with the `unset` function.

If you want to be informed when an unbound variable is referenced, use the `error_reporting` function to change the error-reporting level of the PHP interpreter to `15`. The following call is placed at the beginning of the script in the document file:

```
error_reporting(15);
```

The default error-reporting level is `7`, which does not require the interpreter to report the use of an unbound variable.

11.4.2 Integer Type

PHP has a single integer type, named integer. This type corresponds to the `long` type of C and its successors, which means its size is that of the word size of the machine on which the program is run. In most cases, this is 32 bits, or a bit less (not fewer) than ten decimal digits.

11.4.3 Double Type

PHP's double type corresponds to the `double` type of C and its successors. Double literals can include a decimal point, an exponent, or both. The exponent has the usual form of an `E` or an `e`, followed by a possibly signed integer literal. There does not need to be any digits before or after the decimal point, so both `.345` and `345.` are legal double literals.

---

3. Variables are sometimes declared to have nondefault scopes or lifetimes, as discussed in Section 11.8.

11.4.4 String Type

Characters in PHP are single bytes. (UNICODE is not supported.) There is no character type. A single character data value is represented as a string of length 1.

String literals are defined with either single (') or double quotes (") delimiters. In single-quoted string literals, escape sequences, such as `\n`, are not recognized as anything special, and the values of embedded variables are not substituted. (This substitution is called *interpolation.*) In double-quoted string literals, escape sequences are recognized, and embedded variables are replaced by their current values. For example, the value of

```
'The sum is: $sum'
```

is exactly as it is typed. However, assuming the current value of `$sum` is `10.2`, the value of

```
"The sum is: $sum"
```

is

```
The sum is: 10.2
```

If a double-quoted string literal includes a variable name but you do not want it interpolated, precede the first character of the name (the dollar sign) with a backslash (\). If the name of a variable that is not set to a value is embedded in a double-quoted string literal, the name is replaced by the empty string.

Double-quoted strings can include embedded newline characters that are created by the Enter key. Such characters are exactly like those that result from typing `\n` in the string.

The length of a string is limited only by the available memory on the computer.

11.4.5 Boolean Type

The only two possible values for the Boolean type are `TRUE` and `FALSE`, both of which are case insensitive. Although Boolean is a data type in the same sense as integer, expressions of other types can be used in Boolean context. If you use a non-Boolean expression in Boolean context, you obviously must know how it will be interpreted.

If an integer expression is used in Boolean context, it evaluates to `FALSE` if it is zero; otherwise, it is `TRUE`.

If a string expression is used in Boolean context, it evalutes to `FALSE` if it is either the empty string or the string `"0"`; otherwise, it is `TRUE`. This implies that the string `"0.0"` evaluates to `TRUE`.

The only double value that is interpreted as `FALSE` is exactly `0.0`. Because of rounding errors, as well as the fact that the string `"0.0"` evaluates to `TRUE`, it is not a good idea to use expressions of type double in Boolean context. A value can be very close to zero, but because it is not exactly zero, it will evaluate to `TRUE`.

11.4.6 Arithmetic Operators and Expressions

PHP has the usual (for C-based programming languages) collection of arithmetic operators (+, -, *, /, %, ++, and --) with the usual meanings. In the cases of +, -, and *, if both operands are integers, the operation is integer and an integer result is produced. If either operand is a double, the operation is double and a double result is produced. Division is treated the same way, except that if integer division is done and the result is not an integral value, the result is returned as a double. Any operation on integers that results in integer overflow also produces a double. The operands of the modulus operator (%) are expected to be integers. If one or both are not, they are coerced to integers.

PHP has a large number of predefined functions that operate on numeric values. Some of the most useful of these are shown in Table 11.2. In this table, "number" means either integer or double.

Table 11.2 Some useful predefined functions

Function	Parameter Type	Returns
`floor`	Double	Largest integer less than or equal to the parameter
`ceil`	Double	Smallest integer greater than or equal to the parameter
`round`	Double	Nearest integer
`srand`	Integer	Initializes a random number generator with the parameter
`rand`	Two numbers	A pseudorandom number greater than the first parameter and smaller than the second
`abs`	Number	Absolute value of the parameter
`min`	One or more numbers	Smallest
`max`	One or more numbers	Largest

The other predefined functions for number values are for doing number base conversion and computing exponents, logarithms, and trigonometric functions.

11.4.7 String Operations

The only string operator is the catenation operator, specified with a period (.).

String variables can be treated somewhat like arrays for access to individual characters. The position of a character in a string, relative to zero, can be specified in braces immediately after the variable's name. For example, if `$str` has the value `"apple"`, `$str{3}` is `"l"`.

PHP includes many functions that operate on strings. Some of the most commonly used of these are described in Table 11.3.

Table 11.3 Some commonly used string functions

Function	Parameter Type	Returns
`strlen`	A string	The number of characters in the string
`strcmp`	Two strings	Zero if the two strings are identical, a negative number if the first string belongs before the second (in the ASCII sequence), or a positive number if the second string belongs before the first
`strpos`	Two strings	The character position in the first string of the first character of the second string, if the second string is in the first string; `false` if it is not there
`substr`	A string and an integer	The substring of the string parameter, starting from the position indicated by the second parameter; if a third parameter is given (an integer), it specifies the length of the returned substring
`chop`	A string	The parameter with all whitespace characters removed from its end
`trim`	A string	The parameter with all whitespace characters removed from both ends
`ltrim`	A string	The parameter with all whitespace characters removed from its beginning
`strtolower`	A string	The parameter with all uppercase letters converted to lowercase
`strtoupper`	A string	The parameter with all lowercase letters converted to uppercase

Note for `strpos`: Because `false` is interpreted as zero in numeric context, this can be a problem. To avoid it, compare the returned value to zero using the `===` operator (see Section 11.6.1) to determine whether the match was at the beginning of the first string parameter (or there was no match).

Consider the following example of the use of a string function:

```
$str = "Apples are good";
$sub = substr($str, 7, 1);
```

The value of `$sub` is now `'a'`.

11.4.8 Scalar Type Conversions

PHP, like most other programming languages, includes both implicit and explicit type conversions. Implicit type conversions are called *coercions*. In most

cases, the context of an expression determines the type that is expected or required. The context can cause a coercion of the type of the value of the expression. We have already discussed some of the coercions that take place between the integer and double types and between Boolean and other scalar types. There are also frequent coercions between numeric and string types. Whenever a numeric value appears in string context, the numeric value is coerced to a string. Likewise, whenever a string value appears in numeric context, the string value is coerced to a numeric value. If the string contains a period, an `e`, or an `E`, it is converted to double; otherwise, it is converted to an integer. If the string does not begin with a sign or a digit, the conversion fails and zero is used. Nonnumeric characters following the number in the string are ignored.

When a double is converted to an integer, the fractional part is dropped; rounding is not done.

Explicit type conversions can be specified in three different ways. Using the syntax of C, an expression can be cast to a different type. The cast is a type name in parentheses preceding the expression. For example, if the value of `$sum` is `4.777`, the following produces `4`:

```
(int)$sum
```

Another way to specify explicit type conversion is to use one of the functions `intval`, `doubleval`, or `strval`. For example, if `$sum` is still `4.777`, the following call returns `4`:

```
intval($sum)
```

The third way to specify an explicit type conversion is the `settype` function, which takes two parameters: a variable and a string that specifies a type name. For example, if `$sum` is still `4.777`, the following statement converts the value of `$sum` to `4` and its type to integer:

```
settype($sum, "integer");
```

The type of the value of a variable can be determined in two different ways, the first of which is the `gettype` function. The `gettype` function takes a variable as its parameter and returns a string that has the name of the type of the current value of the variable. One possible return value of `gettype` is `"unknown"`. The other way to determine the type of the value of a variable is to use one or more of the type-testing functions, each of which takes a variable name as a parameter and returns a Boolean value. These are `is_int`, `is_integer`, and `is_long`, which test for integer type; `is_double`, `is_float`, and `is_real`, which test for double type; `is_bool`, which tests for Boolean type; and `is_string`, which tests for string type.[4]

4. PHP also has the `is_array` function to test for arrays and the `is_object` function to test for objects.

11.4.9 Assignment Operators

PHP has the same set of assignment operators as its predecessor languages, C and Perl, including the compound assignment operators such as `+= and /=`.

11.5 Output

Any output from a PHP script becomes part of the document the PHP processor is building. Therefore, all output must be in the form of XHTML, which may include embedded client-side script.

The `print` function[5] is used to create simple unformatted output. It can be called with or without parentheses around its parameter. For example, the following statement is legal:

```
print "Apples are red <br /> Kumquats aren't <br />";
```

Although `print` expects a string parameter, if some other type value is given, the PHP interpreter will coerce it to a string without complaint. For example, the following statement will produce `47`:

```
print(47);
```

Because variables that appear in double-quoted strings are interpolated, it is easy to label output. For example,

```
print "The result is: $result <br />";
```

PHP borrows the `printf` function from C (also copied into Perl). It is used when complete control over the format of displayed data is required. The general form of a call to `printf` is as follows:

`printf(`*literal_string*`,` *param1*`,` *param2*`, ...)`

The literal string can include labeling information about the parameters whose values are to be displayed. It also contains format codes for those values. The form of the format codes is a percent sign (`%`) followed by a field width and a type specifier. The most common type specifiers are `s` for strings, `d` for integers, and `f` for floats and doubles. The field width is either an integer literal (for integers) or two integer literals separated by a decimal point for floats and doubles. The integer literal to the right of the decimal point specifies the number of digits to be displayed to the right of the decimal point. Consider the following examples:

`%10s`—a character string field of 10 characters
`%6d`—an integer field of 6 digits
`%5.2f`—a float or double field of 8 spaces, with two digits to the right of the decimal point, the decimal point, and 5 digits to the left

5. PHP also has the `echo` function, which is similar to `print`.

The position of the format code in the first parameter to `printf` indicates the place in the output where the associated value should appear. For example, condider the following:

```
$day = "Tuesday";
$high = 79;
printf("The high on %7s was %3d", $day, $high);
```

Note that `printf` requires parentheses around its parameters.

The following simple example displays a welcome message and the current day of the week, the month, and day of the month. The date information is generated with the `date` function, whose first parameter is a string that specifies the parts of the date you want to see. In our example, `l` requests the day of the week, `F` requests the month, `j` requests the day of the week, and an `S` next to the `j` gets the correct suffix for the day (for example, `st` or `nd`). The details of `date` can be found at `http://www.php.net`. Figure 11.1 shows a display of the output of `today.php`.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1 //EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- today.php - A trivial example to illustrate a php document -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> today.php </title>
  </head>
  <body>
    <p>
      <?php
        print "<b>Welcome to my home page <br /> <br />";
        print "Today is:</b> ";
        print date("l, F jS");
        print "<br />";
      ?>
    </p>
  </body>
</html>
```

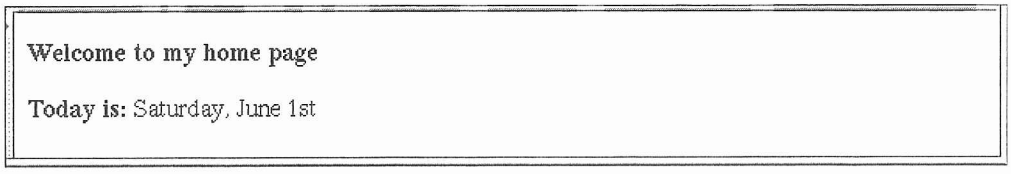


Figure 11.1 Display of the output of `today.php`

11.6 Control Statements

The control statements of PHP are not remarkable—in fact, they are very similar to those of C and its descendants. The control expression used in PHP's control statements can be any type. The interpreter evaluates the control expression and, in the cases of `if` and loop statements, coerces the resulting value, if necessary, to Boolean.

11.6.1 Relational Operators

PHP uses the eight relational operators of JavaScript. The usual six (>, <, >=, <=, `!=`, and ==) have the usual meanings. It also has ===, which produces `TRUE` only if both operands are the same type and have the same value, and `!==`, the opposite of ===. If the types of the operands of the other six relational operators are not the same, one is coerced to the type of the other. If a string is compared to a number and the string can be converted to a number (it is in fact a string version of a number, for example `"42"`), the string will be converted and a numeric comparison will be done. If the string cannot be converted to a number, the numeric operand will be converted to a string, and a string comparison will be done. If both operands are strings that can be converted to numbers, both will be converted and a numeric comparison will be done. This is often not what you want. To avoid this and similar problems associated with string-to-number coercions, if either or both operands are strings that could be converted to numbers, the `strcmp` function should be used rather than one of the comparison operators.

11.6.2 Boolean Operators

There are six Boolean operators: `and`, `or`, `xor`, `!`, `&&`, and `||`. The `and` and `&&` operators perform the same operation, as do `or` and `||`. The difference between these is that the precedence of `and` and `or` is lower than that of `&&` and `||`. All of PHP's binary Boolean operators are evaluated as short-circuit operators.

11.6.3 Selection Statements

PHP's `if` statement is like that of C. The control expression can be an expression of any type, but its value is coerced to Boolean. The controlled statement segment can be either an individual statement or a compound statement. An `if` statement can include any number of `elseif` clauses. Following is a simple example of an `if` construct:

```
if ($day == "Saturday" || $day == "Sunday")
  $today = "weekend";
else {
  $today = "weekday";
  $work = true;
}
```

The `switch` statement has the form and semantics of that of JavaScript. The type of the control expression and the `case` expressions is either integer, double, or string. If necessary, the values of the `case` expressions are coerced to the type of the control expression for the comparisons. A `default` case can be included. As with its ancestor in C and Java, a `break` statement must follow each selectable segment if control is not to flow to the following segment. Following is a simple example of a `switch` construct:

```
switch ($bordersize) {
  case "0": print "<table>";
                  break;
  case "1": print "<table border = '1'>";
              break;
  case "4": print "<table border = '4'>";
              break;
  case "8": print "<table border = '8'>";
              break;
  default: print "Error-invalid value: $bordersize <br />";
}
```

11.6.4 Loop Statements

The `while`, `for`, and `do-while` statements of PHP are exactly like those of JavaScript. PHP also has a `foreach` statement, which is discussed in Section 11.7.4. The following example computes the factorial of `$n`:

```
$fact = 1;
$count = 1;
while ($count < $n) {
  $count++;
  $fact *= $count;
}
```

The following example computes the sum of the positive integers up to `100`:

```
$count = 1;
$sum = 0;
do {
  $sum += $count;
  $count++;
} while ($count <= 100);
```

The following example computes the factorial of `$n`:

```
for ($count = 1, $fact = 1; $count < $n;) {
  $count++;
  $fact *= $count;
}
```

The `break` statement can be used to terminate the execution of a `for`, `foreach`, `while`, or `do-while` construct. The `continue` statement is used in loop constructs to skip the remainder of the current iteration but continue execution at the beginning of the next.

11.6.5 An Example

The following example is meant to illustrate the form of an XHTML/PHP document, as well as some simple mathematical functions and the intermingling of XHTML and PHP in a document. The `sqrt` function returns the square root of its parameter; the `pow` function raises its first parameter to the power of its second parameter.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1 //EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- powers.php
     An example to illustrate loops and arithmetic
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> powers.php </title>
  </head>
  <body>
    <table border = "border">
      <caption> Powers table </caption>
      <tr>
        <th> Number </th>
        <th> Square Root </th>
        <th> Square </th>
        <th> Cube </th>
        <th> Quad </th>
      </tr>
      <?php
        for ($number = 1; $number <=10; $number++) {
          $root = sqrt($number);
          $square = pow($number, 2);
          $cube = pow($number, 3);
          $quad = pow($number, 4);
          print("<tr align = 'center'> <td> $number </td>");
          print("<td> $root </td> <td> $square </td>");
          print("<td> $cube </td> <td> $quad </td> </tr>");
        }
```

```
    ?>
  </table>
 </body>
</html>
```

Figure 11.2 displays the output of `powers.php`.

Powers table

Number	Square Root	Square	Cube	Quad
1	1	1	1	1
2	1.4142135623731	4	8	16
3	1.7320508075689	9	27	81
4	2	16	64	256
5	2.2360679774998	25	125	625
6	2.4494897427832	36	216	1296
7	2.6457513110646	49	343	2401
8	2.8284271247462	64	512	4096
9	3	81	729	6561
10	3.1622776601684	100	1000	10000

Figure 11.2 The output of `powers.php`

11.7 Arrays

Arrays in PHP are unlike those of any other common programming language. They are best described as a combination of the arrays of a typical language and associative arrays, or hashes, found in some other languages such as Perl, Ruby, and Python. This makes them the ultimate in flexible, built-in data structures. Each array element consists of two parts, a key and a value. If the array has a logical structure that is similar to an array in another language, the keys just happen to be non-negative integers and are always in ascending order. If the array has a logical structure that is similar to a hash, its keys are strings, and the order of its elements is determined with a system-designed hashing function. The string keys of a PHP array are sometimes people's names; sometimes they are the names of the days of the week. They are always a collection of strings of some significance. One interesting thing about PHP arrays is that they can have some elements with integer keys and some with string keys.

11.7.1 Array Creation

There are two ways to create an array in PHP. The assignment operation creates scalar variables. The same operation works for arrays—assigning a value to an element of an array that does not yet exist creates the array. For example, assuming no array named `$list` currently exists, the following statement creates one:

```
$list[0] = 17;
```

If the script has a scalar variable named `$list` prior to this assignment, `$list` is now an array. If empty brackets are used in an assignment to an array, a numeric subscript is implicitly furnished. The furnished subscript is 1 greater than the largest used so far in the array, if the array already has elements with numeric keys. If the array currently has no elements with numeric keys, the value `0` is used. For example, in the following code, the second element's subscript will be `2`:

```
$list[1] = "Today is my birthday!";
$list[] = 42;
```

This example also shows that the elements of an array do not need to have the same type.

The second way to create an array is with the `array` construct. We call this a construct because, although the syntax of using it is the same as that of a function call, it is not a function. The parameters to `array` specify the values to be placed in a new array and sometimes also the keys. If the array is like a traditional array, only the values need to be specified. (The PHP interpreter will furnish the numeric keys.) For example:

```
$list = array(17, 24, 45, 91);
```

This assignment creates a traditional array of four elements, with the keys `0`, `1`, `2`, and `3`. If you would rather have different keys, they can be specified in the array construct, as shown in the following:

```
$list = array(1 => 17, 2 => 24, 3 => 42, 4 => 91);
```

An array construct with empty parentheses creates an empty array. For example, in the following statement, `$list` becomes a variable whose value is an array with no elements:

```
$list = array();
```

The following statement creates an array that has the form of a hash:

```
$ages = array("Joe" => 42, "Mary" => 41, "Bif" => 17);
```

Some built-in functions return arrays. For example, some of the functions that access databases return arrays.

PHP arrays do not need to be purely in the form of traditional arrays or hashes; they can be mixtures of both. For example, we could have the following:

```
$stuff = array("make" => "Cessna", "model" => "C210",
               "year" => 1960, 3 => "sold");
```

11.7.2 Accessing Array Elements

Individual array elements can be accessed by subscripting, as in other programming languages. The value in the subscript, which is enclosed in brackets, is the key of the value being referenced. The same brackets are used regardless of whether the key is a number or a string. For example, the value of the element whose key is `"Mary"` in the `$ages` array can be set to `29` with the following statement:

```
$ages['Mary'] = 29;
```

Multiple elements of an array can be assigned to scalar variables in one statement, using the `list` construct. This is similar to the list assignments of Perl. For example:

```
$trees = array("oak" , "pine", "binary");
list($hardwood, $softwood, $data_structure) = $trees;
```

In this example, `$hardwood`, `$softwood`, and `$data_structure` are set to `"oak"`, `"pine"`, and `"binary"`, respectively.

11.7.3 Functions for Dealing with Arrays

A whole array can be deleted with `unset`, as with a scalar variable. Individual elements of an array also can be removed with `unset`, as in the following:

```
$list = array(2, 4, 6, 8);
unset($list[2]);
```

Now `$list` has three remaining elements with keys `0`, `1`, and `3` and elements `2`, `4`, and `8`.

The collection of keys and the collection of values of an array can be extracted with built-in functions. The `array_keys` function takes an array as its parameter and returns an array of the keys of the given array. The returned array uses `0`, `1`, and so forth as its keys. The `array_values` function does for values what `array_keys` does for keys. For example:

```
$highs = array("Mon" => 74, "Tue" => 70, "Wed" => 67,
               "Thu" => 62, "Fri" => 65);
$days = array_keys($highs);
$temps = array_values($highs);
```

Now the value of `$days` is `("Mon", "Tue", "Wed", "Thu", "Fri")`, and the value of `$temps` is `(74, 70, 67, 62, 65)`. In both cases, the keys are `(0, 1, 2, 3, 4)`.

The existance of an element of a specific key can be determined with the `array_key_exists` function, which returns a Boolean value. For example, consider the following:

```
$highs = array("Mon" => 74, "Tue" => 70, "Wed" => 67,
               "Thu" => 62, "Fri" => 65);
if (array_key_exists("Tue", $highs)) {
  $tues_high = $highs["Tue"];
  print "The high on Tuesday was $tues_high <br />";
}
```

Note that PHP does not interpolate array elements embedded in double-quoted strings. That is the reason for the assignment statement in the `if` construct above. An array name embedded in a double-quoted string results in the word `Array` being inserted in the string in place of the array's name.

The `is_array` function is similar to the `is_int` function: It takes a variable as its parameter and returns `TRUE` if the variable is an array, `FALSE` otherwise. The `in_array` function takes two parameters—an expression and an array—and returns `TRUE` if the value of the expression is a value in the array; otherwise, it returns `FALSE`.

The number of elements in an array can be determined with the `sizeof` function. For example, consider the following code:

```
$list = array("Bob", "Fred", "Alan", "Bozo");
$len = sizeof($list);
```

After executing this code, `$len` will be `4`.

It is often convenient to be able to convert between strings and arrays. These conversions can be done with the `implode` and `explode` functions. The `explode` function explodes a string into substrings and returns them in an array. The delimiters of the substrings are defined by the first parameter to `explode`, which is a string; the second parameter is the string to be converted. For example, consider the following:

```
$str = "April in Paris, Texas is nice";
$words = explode(" ", $str);
```

Now `$words` contains `("April", "in", "Paris,", "Texas", "is", "nice")`.

The `implode` function does the inverse of `explode`. Given a separator character (or string) and an array, it catenates the elements of the array together, using the given separator string between the elements, and returns the result as a string. For example:

```
$words = array("Are", "you", "lonesome", "tonight");
$str = implode(" ", $words);
```

Now `$str` has `"Are you lonesome tonight"` (which is obviously a rhetorical question).

Internally, the elements of an array are stored in a linked list of cells, where each cell includes both the key and the value of the element. The cells themselves are stored in memory through a key hashing function so that they are randomly distributed in a reserved block of storage. Accesses to elements through

string keys are implemented through the hashing function. However, the elements all have links that connect them in the order in which they were created, which allows them to be accessed in that order if the keys are strings and in the order of their keys if the keys are numbers. Section 11.7.4 discusses the ways array elements can be accessed in order.

Figure 11.3 shows the internal logical structure of an array. Although arrays may not be implemented in this exact way, it shows how the two different access methods could be supported.

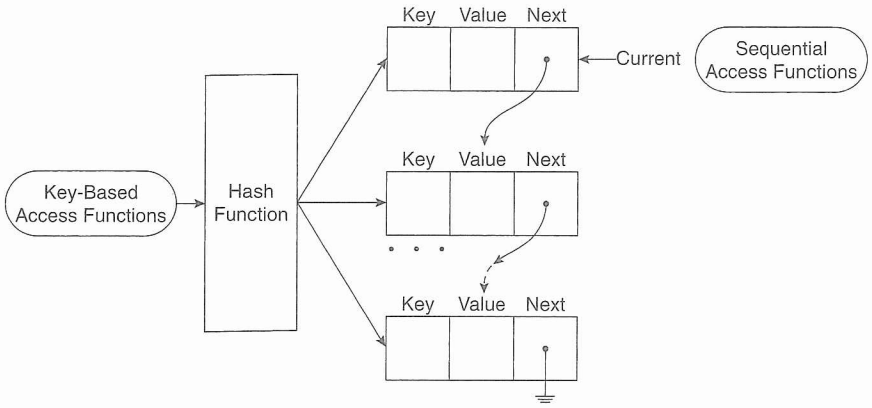


Figure 11.3 Logical internal structure of arrays

11.7.4 Sequential Access to Array Elements

PHP includes several different ways to access array elements in sequential order. Every array has an internal pointer, or marker, that references one element of the array. We call this the "current" pointer. This pointer is initialized to reference the first element of the array at the time the array is created. The element being referenced by the pointer can be obtained with the `current` function. For example, consider the following code:

```
$cities = array("Hoboken", "Chicago", "Moab", "Atlantis");
$city = current($cities);
print("The first city is $city <br />");
```

This code produces the following:

```
The first city is Hoboken
```

The "current" pointer can be moved with the `next` function, which both moves the pointer to the next array element and returns the value of that element. If the "current" pointer is already pointing at the last element of the array, `next` returns `FALSE`. For example, if the "current" pointer is referencing the first element of the `$cities` array, the following code produces a list of all of the elements of that array:

```
$city = current($cities);
print("$city <br />");
while ($city = next($cities))
  print("$city <br />");
```

One problem with using the `next` function for loop control, as shown in the preceding example, occurs when the array includes an element with the value `FALSE`. The loop ends, but not because the "current" pointer ran off the end of the array. The `each` function, which returns a two-element array consisting of the key and the value of the "current" element, avoids this problem. It returns `FALSE` only if the "current" pointer has gone past the last element of the array. The keys of the two elements of the return value from `each` are the strings `"key"` and `"value"`. Another difference between `each` and `next` is that `each` returns the element being referenced by the "current" pointer and then moves that pointer. The `next` function first moves the "current" pointer and then returns the value being referenced by the "current" pointer. As an example of the use of `each`, consider the following code:

```
$salaries = array("Mike" => 42500, "Jerry" => 51250,
                  "Fred" => 37920);
while ($employee = each($salaries)) {
  $name = $employee["key"];
  $salary = $employee["value"];
  print("The salary of $name is $salary <br />");
}
```

The output produced by this code is as follows:

```
The salary of Mike is 42500
The salary of Jerry is 51250
The salary of Fred is 37920
```

The "current" pointer can be moved backward (that is, to the element before the "current" element) with the `prev` function. Like the `next` function, the `prev` function returns the value of the element referenced by the "current" pointer after the pointer has been moved. The "current" pointer can be set to the first element with the `reset` function, which also returns the value of the first element. It can be set to the last element of the array with the `end` function, which also returns the value of the last element.

The `key` function, when given the name of an array, returns the key of the "current" element of the array.

The `array_push` and `array_pop` functions provide a simple way to implement a stack in an array. The `array_push` function takes as its first parameter an array. After this first parameter, there can be any number of additional parameters. The values of all subsequent parameters are placed at the end of the array. The `array_push` function returns the new number of elements in the array. The `array_pop` function takes a single parameter, the name of an array. It removes the last element from the array and returns it. The value `NULL` is returned if the array is empty.

The `foreach` statement is designed to build loops that process all of the elements of an array. This statement has two forms:

`foreach (`*array* `as` *scalar_variable*`)` *loop body*
`foreach (`*array* `as` *key* `=>` *value*`)` *loop body*

In the first form, one of the array's values is set to the scalar variable for each iteration of the loop body. The "current" pointer is implicitly initialized, as with `reset`, before the first iteration. For example:

```
foreach ($list as $temp)
  print("$temp <br />");
```

This code will produce the values of all of the elements of `$list`.

The second form of `foreach` provides both the key and the value of each element of the array. For example:

```
$lows = array("Mon" => 23, "Tue" => 18, "Wed" => 27);
foreach ($lows as $day => $temp)
  print("The low temperature on $day was $temp <br />");
```

11.7.5 Sorting Arrays

The `sort` function, which takes an array as a parameter, sorts the values in the array, replacing the keys with the numeric keys, `0`, `1`, `2`, ... The array can have both string and numeric values. The string values migrate to the beginning of the array in alphabetical order. The numeric values follow in ascending order. Regardless of the types of the keys in the original array, the sorted array has `0`, `1`, `2`, and so forth as keys. This function is obviously meant for sorting traditional arrays of either strings or numbers. Although it causes no problems, it seems to be a rare situation in which one would plan on sorting arrays with both strings and numbers as values.

The `asort` function is used to sort arrays that correspond to Perl hashes. It sorts the elements of a given array by their values but keeps the original key/value associations. As with `sort`, string values all appear before the numeric values, in alphabetical order. The numeric values follow in ascending order.

The `ksort` function sorts its given array by keys, rather than values. The key/value associations are maintained by the process.

The `rsort`, `arsort`, and `krsort` functions behave like the `sort`, `asort`, and `ksort` functions, respectively, except that they sort into the reverse orders of their counterparts.

The following example illustrates `sort`, `asort`, and `ksort`.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- sorting.php - An example to illustrate several of the
```

```
     sorting functions -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Sorting </title>
  </head>
  <body>
    <?php
      $original = array("Fred" => 31, "Al" => 27,
                        "Gandalf" => "wizard",
                        "Betty" => 42, "Frodo" => "hobbit");
    ?>
    <h4> Original Array </h4>
    <?php
      foreach ($original as $key => $value)
        print("[$key] => $value <br />");

      $new = $original;
      sort($new);
    ?>
    <h4> Array sorted with sort </h4>
    <?php
      foreach ($new as $key => $value)
        print("[$key] = $value <br />");

      $new = $original;
      asort($new);
    ?>
    <h4> Array sorted with asort </h4>
    <?php
      foreach ($new as $key => $value)
        print("[$key] = $value <br />");

      $new = $original;
      ksort($new);
    ?>
    <h4> Array sorted with ksort </h4>
    <?php
      foreach ($new as $key => $value)
        print("[$key] = $value <br />");
    ?>
  </body>
</html>
```

Figure 11.4 shows the output of `sorting.php`.

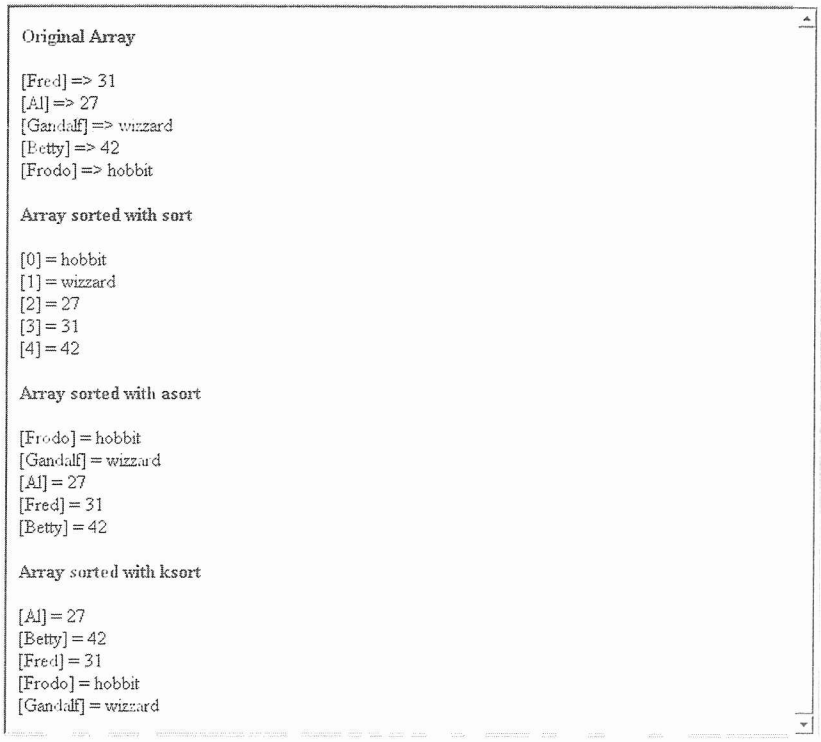


Figure 11.4 The output of `sorting.php`

We have now discussed just a few of the most useful built-in functions for arrays. PHP has 57 such functions, so most remain unmentioned.

11.8 Functions

PHP supports user-defined functions that are typical for C-based programming languages.

11.8.1 General Characteristics of Functions

The general form of a PHP function definition is as follows:

```
function name([parameters]) {
  ...
}
```

The square brackets around the parameters mean that they are optional. Because a function's definition does not need to appear in a document before the function is called, the placement of function definitions in a document is, strictly speaking, irrelevant. If a second definition of a function appears in a script, it is reported as an error because function overloading is not allowed and functions cannot be redefined. Function definitions can be nested, as they can in JavaScript. However, because we do not believe the benefit of nested functions is worth the additional complexity they bring to scripts that use them, they are not discussed in this chapter.

Remember that function names are not case sensitive. So, you cannot have a function named `sum` and another named `Sum`. The PHP interpreter will see them as the same function and issue an error message stating that you have two definitions for the same function.

The `return` statement is used in a function to specify the value to be returned to the caller. Function execution ends when a `return` statement is encountered or the last statement in the function has been executed. In either case, control returns to the caller. If no `return` statement was executed, no value is returned.

If one or more related functions are used by more than one document, it is convenient to store their definitions in a separate file and copy that file into those documents when they are requested by a client (browser). This is done using the `include` function, which was described in Section 11.3.

11.8.2 Parameters

As always, we call the parameters in the call to a function *actual parameters*. We call the parameters that are listed in the function definition *formal parameters*. An actual parameter can be any expression. A formal parameter must be a variable name.

The number of actual parameters in a call to a function does not need to match the number of formal parameters defined in that function. If there are too few actual parameters in a call, the corresponding formal parameters will be unbound variables. If there are too many actual parameters, the excess parameters will be ignored. The absence of a requirement for matching numbers of parameters allows the language to support functions with a variable number of parameters.

The default parameter-passing mechanism of PHP is pass by value. This means that, in effect, the values of actual parameters are copied into the memory locations associated with the corresponding formal parameters in the called function. The values of the formal parameters are never copied back to the caller, so passing by value implements one-way communication to the function. This is the most commonly needed mechanism for parameter passing. Consider the following function definition:

```
function max_abs($first, $second) {
  $first = abs($first);
  $second = abs($second);
  if ($first >= $second)
    return $first;
  else
    return $second;
   }
```

This function returns the largest absolute value of the two given numbers. Although it potentially changes both of its parameters, the actual parameters in the caller are unchanged (because they were passed by value).

Sometimes parameters that provide two-way communication between the caller and the function are needed—for example, so a function can return more than one value. One common way to provide two-way communication is to pass the address of the actual parameter, rather than its value, to the function. Then, when the formal parameter is changed (in the function), it also changes the corresponding actual parameter. Such parameters are said to be passed by reference.

Pass-by-reference parameters can be specified in PHP in two ways. One way is to add an ampersand (&) to the beginning of the name of the formal parameter that you want to be passed by reference. Of course, passing by reference only makes sense if the actual parameter is a variable. The other way is to add an ampersand to the actual parameter in the function call. These two techniques have identical semantics. Consider the following example:

```
function set_max(&$max, $first, $second) {
  If ($first >= $second)
    $max = $first;
  else
    $max = $second;
}
```

In this example, the first actual parameter in the caller is set to the larger of the second and third parameters.

11.8.3 The Scope of Variables

The default scope of a variable defined in a function is local. If a variable defined in a function has the same name as a variable used outside the function, there is no interference between the two. A local variable is visible only in the function in which it is used. For example, consider the following example:

```
function summer($list) {
  $sum = 0;
  foreach ($list as $value)
    $sum += $value;
  return $sum;
}
$sum = 10;
$nums = array(2, 4, 6, 8);
$ans = summer($nums);
print "The sum of the values in \$nums is: $ans <br />";
print "The value of \$sum is still: $sum <br />";
```

The output of this code is as follows:

```
The sum of the values in $nums is: 20
The value of $sum is still: 10
```

This output shows that the value of `$sum` in the calling code is not affected by the use of the local variable `$sum` in the function. The purpose of the design of local variables is simple: A function should behave the same way regardless of the context of its use. Furthermore, when naming a variable while designing a function, the author should not need to worry about conflicts with the names of variables used outside the function.

In some cases, it is convenient for the code in a function to be able to access a variable that is defined outside the function. For this situation, PHP has the `global` declaration. When a variable is listed in a `global` declaration in a function, that variable is expected to be defined outside the function. So, such a variable has the same meaning inside the function as outside. For example, consider the following code:

```
$big_sum = 0;
...
/* Function summer
   Parameter: An array of integers
   Returns: The sum of the elements of the parameter
            array
   Side effect: Add the computed sum to the global,
                $big_sum
*/
function summer ($list) {
  global $big_sum;   //** Get access to $big_sum
  $sum = 0;
  foreach ($list as $value)
    $sum += $value;
  $big_sum += $sum;
  return $sum;
}  //** end of summer
```

```
...
$ans1 = summer($list1);
$ans2 = summer($list2);
...
print "The sum of all array elements is: $big_sum <br />";
```

If the `global` declaration were not included in the function, the script would have two variables named `$big_sum`, the global one and the one that is local to the function. Without the declaration, this script cannot do what it meant to do.

11.8.4 The Lifetime of Variables

In some situations, a function must be history sensitive; that is, it must retain information about previous activations. The default lifetime of local variables in a PHP function is from the time the variable is first used (that is, when storage for it is allocated) until the function's execution terminates. To support history sensitivity, a function must have static local variables. The lifetime of a static variable in a function begins when the variable is first used in the first execution of the function. Its lifetime ends when the script execution ends. In the case of PHP, this is when the browser leaves the document in which the PHP script is embedded.

In PHP a local variable in a function can be specified to be static by declaring it with the `static` reserved word. Such a declaration can include an initial value, which is only assigned the first time the declaration is reached. For example, consider the following function:

```
function do_it ($param) {
  static $count = 0;
  count++;
  print "do_it has now been called $count times <br />";
  ...
}
```

This function displays the number of times it has been called, even if it is called from several different places. The fact that its local variable, `$count`, is static allows this to be done.

11.9 Pattern Matching

PHP includes two different kinds of string pattern matching using regular expressions: one that is based on POSIX regular expressions and one that is based on Perl regular expressions. The POSIX regular expressions are compiled into PHP, but the Perl-Compatible Regular Expression (PCRE) library must be compiled before Perl regular expressions can be used. A detailed discussion of PHP pattern matching is beyond the scope of this chapter. Furthermore, Perl-style regular expressions are described in Sections 4.12.1 to 4.12.3. Therefore, we provide only a brief description of a single PHP function for pattern matching in this section.

The `preg_match`[6] function takes two parameters, the first of which is the Perl-style regular expression as a string. The second parameter is the string to be searched. For example:

```
if (preg_match("/^PHP/", $str))
  print "\$str begins with PHP <br />";
else
  print "\$str does not begin with PHP <br />";
```

The `preg_split` function operates on strings but returns an array and uses patterns, so it is discussed here rather than with the other string functions in Section 11.4.7. `preg_split` takes two parameters, the first of which is a Perl-style pattern as a string. The second parameter is the string to be split.[7] For example, consider the following sample code:

```
$fruit_string = "apple : orange : banana";
$fruits = preg_split("/ : /", $fruit_string);
```

The array `$fruits` now has `("apple", "orange", "banana")`.

The following example illustrates the use of `preg_split` on text to parse out the words and produce a frequency-of-occurrence table:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1 //EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!--    word_table.php
        Uses a function to split a given string of text into
        its constituent words. It also determines the frequency of
        occurrence of each word. The words are separated by
        whitespace or punctuation, possibly followed by whitespace.
        The punctuation can be a period, a comma, a semicolon, a
        colon, an exclamation point, or a question mark.
        -->
<html xmlns = "http://www.w3.org/1999/xhtml">
<head> <title> word_table.php </title>
</head>
<body>
<?php

// Function splitter
//  Parameter: a string of text containing words and punctuation
//  Returns: an array in which the unique words of the string are
```

6. The first part of the name, `preg`, is an acronym for *P*erl *reg*ular, which indicates the style of regular expression used.

7. The PHP `preg_split` function is exactly like the Perl `split` function.

```
//           the keys and their frequencies are the values.
function splitter($str) {

// Create the empty word frequency array
  $freq = array();

// Split the parameter string into words
  $words = preg_split("/[\.,;:!\?]\s*/", $str);

// Loop to count the words (either increment or initialize to 1)
  foreach ($words as $word) {
    $keys = array_keys($freq);
    if(in_array($word, $keys))
      $freq[$word]++;
    else
      $freq[$word] = 1;
  }
  return $freq;
} #** End of splitter

// Main test driver
  $str = "apples are good for you, or don't you like apples?
          or maybe you like oranges better than apples";

// Call splitter
  $tbl = splitter($str);

// Display the words and their frequencies
  print "<br /> Word Frequency <br /><br />";
  $sorted_keys = array_keys($tbl);
  sort($sorted_keys);
  foreach ($sorted_keys as $word)
    print "$word $tbl[$word] <br />";
?>
</body>
</html>
```

The output of this script is as follows:

```
Word Frequency

apples 3
are 1
better 1
don't 1
for 1
```

```
good 1
like 2
maybe 1
or 2
oranges 1
than 1
you 3
```

11.10 Form Handling

We discussed form handling using CGI with Perl in Chapter 9, "Using Perl for CGI Programming." Form handling using Java servlets was covered in Chapter 10, "Servlets and Java Server Pages." As you will soon see, form handling with PHP is simpler than either of those two approaches.

It may seem strange, but when PHP is used for form handling, the PHP script is embedded in an XHTML document, like other uses of PHP. Although it is possible to have a PHP script handle form data in the same XHTML document that defines the form, it is perhaps clearer to use two separate documents. For this latter case, the document that defines the form specifies the document that handles the form data in the `action` attribute of its `<form>` tag.

PHP can be configured so that form data values are directly available as implicit variables whose names match the names of the corresponding form elements. However, this is not allowed in many Web servers (through the configuration of PHP) because it entails a security risk. The recommended approach is to use the implicit arrays for form values, `$_POST` and `$_GET`. These arrays have keys that match the form element names and values that were input by the client. For example, if a form has a text box named `phone` and the form method is POST, the value of that element is available in the PHP script as follows:

```
$_POST["phone"]
```

The following is an XHTML document that presents a form for popcorn sales that is nearly identical to the one that appears in Chapter 9.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- popcorn3.html - This describes the popcorn sales form -->
<html xmlns = "http:///www.w3.org/1999/xhtml">
  <head>
    <title> Popcorn Sales - for PHP handling </title>
  </head>
  <body>
    <form action = "http://cs.uccs.edu/~rws/popcorn3.php"
          method = "post">
```

```
      <h2> Welcome to Millennium Gymnastics Booster Club Popcorn
           Sales </h2>
      <table>

<!-- Text widgets for the customer's name and address -->
        <tr>
          <td> Buyer's Name: </td>
          <td> <input type = "text" name = "name"
                      size = "30" /></td>
        </tr>
        <tr>
          <td> Street Address: </td>
          <td> <input type = "text" name = "street"
                      size = "30" /></td>
        </tr>
        <tr>
          <td> City, State, Zip: </td>
          <td> <input type = "text" name = "city"
                      size = "30" /></td>
        </tr>
      </table>
      <p />
      <table border = "border">

<!-- First, the column headings -->
        <tr>
          <th> Product </th>
          <th> Price </th>
          <th> Quantity </th>
        </tr>

<!-- Now, the table data entries -->
        <tr>
          <td> Unpopped Popcorn (1 lb.) </td>
          <td> $3.00 </td>
          <td align = "center">
            <input type = "text" name = "unpop"
                   size = "3" /></td>
        </tr>
        <tr>
          <td> Caramel Popcorn (2 lb. canister) </td>
          <td> $3.50 </td>
          <td align = "center">
            <input type = "text" name = "caramel"
                   size = "3" /> </td>
        </tr>
```

```
        <tr>
          <td> Caramel Nut Popcorn (2 lb. canister) </td>
          <td> $4.50 </td>
          <td align = "center">
            <input type = "text" name = "caramelnut"
                   size = "3" /> </td>
        </tr>
        <tr>
          <td> Toffey Nut Popcorn (2 lb. canister) </td>
          <td> $5.00 </td>
          <td align = "center">
            <input type = "text" name = "toffeynut"
                   size = "3" /> </td>
        </tr>
      </table>
      <p />

<!-- The radio buttons for the payment method -->
      <h3> Payment Method </h3>
      <p>
        <input type = "radio" name = "payment" value = "visa"
              checked = "checked" />
          Visa <br />
        <input type = "radio" name = "payment" value = "mc" />
          Master Card <br />
        <input type = "radio" name = "payment"
               value = "discover" />
          Discover <br />
        <input type = "radio" name = "payment" value = "check" />
          Check <br /> <br />

<!-- The submit and reset buttons -->
        <input type = "submit" value = "Submit Order" />
        <input type = "reset" value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

Figure 11.5 shows the display of `popcorn3.html`.

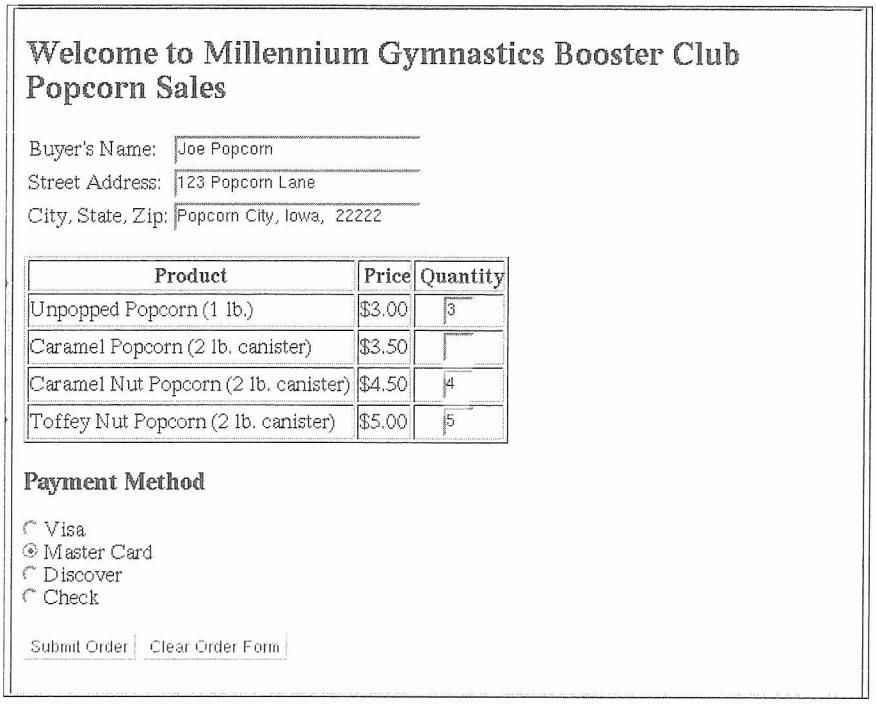


Figure 11.5 The display of `popcorn3.html`

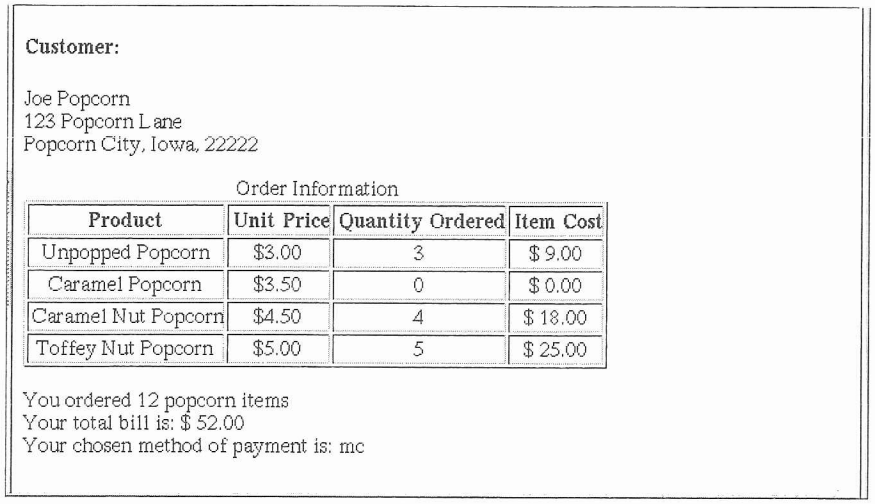
The PHP script that handles the data from the form described in `popcorn3.html` follows. It uses the form data to compute the cost of each product, the total cost of the order, and the total number of ordered items. The product name, unit price, number ordered, and total cost for each product are presented to the client in a table. The table is defined with interwoven XHTML markup and PHP script. The table structure is described with XHTML, but the contents of some of the data cells are defined with PHP.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1 //EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- popcorn3.php - Processes the form described in
     popcorn3.html
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Process the popcorn3.html form </title>
  </head>
```

```
  <body>
    <?php

// Get form data values
      $unpop = $_POST["unpop"];
      $caramel = $_POST["caramel"];
      $caramelnut = $_POST["caramelnut"];
      $toffeynut = $_POST["toffeynut"];
      $name = $_POST["name"];
      $street = $_POST["street"];
      $city = $_POST["city"];
      $payment = $_POST["payment"];

// If any of the quantities are blank, set them to zero
      if ($unpop == "") $unpop = 0;
      if ($caramel == "") $caramel = 0;
      if ($caramelnut == "") $caramelnut = 0;
      if ($toffeynut == "") $toffeynut = 0;

// Compute the item costs and total cost
      $unpop_cost = 3.0 * $unpop;
      $caramel_cost = 3.5 * $caramel;
      $caramelnut_cost = 4.5 * $caramelnut;
      $toffeynut_cost = 5.0 * $toffeynut;
      $total_price = $unpop_cost + $caramel_cost +
                     $caramelnut_cost + $toffeynut_cost;
      $total_items = $unpop + $caramel + $caramelnut + $toffeynut;

// Return the results to the browser in a table
    ?>
    <h4> Customer: </h4>
    <?php
      print ("$name <br /> $street <br /> $city <br />");
    ?>
    <p /> <p />
    <table border = "border">
      <caption> Order Information </caption>
      <tr>
        <th> Product </th>
        <th> Unit Price </th>
        <th> Quantity Ordered </th>
        <th> Item Cost </th>
      </tr>
```

```
      <tr align = "center">
        <td> Unpopped Popcorn </td>
        <td> $3.00 </td>
        <td> <?php print ("$unpop"); ?> </td>
        <td> <?php printf ("$ %4.2f", $unpop_cost); ?>
        </td>
      </tr>
      <tr align = "center">
        <td> Caramel Popcorn </td>
        <td> $3.50 </td>
        <td> <?php print ("$caramel"); ?> </td>
        <td> <?php printf ("$ %4.2f", $caramel_cost); ?>
        </td>
        </tr>
      <tr align = "center">
        <td> Caramel Nut Popcorn </td>
        <td> $4.50 </td>
        <td> <?php print ("$caramelnut"); ?> </td>
        <td> <?php printf ("$ %4.2f", $caramelnut_cost); ?>
        </td>
      </tr>
      <tr align = "center">
        <td> Toffey Nut Popcorn </td>
        <td> $5.00 </td>
        <td> <?php print ("$toffeynut"); ?> </td>
        <td> <?php printf ("$ %4.2f", $toffeynut_cost); ?>
        </td>
      </tr>
    </table>
    <p /> <p />

    <?php
      print "You ordered $total_items popcorn items <br />";
      printf ("Your total bill is: $ %5.2f <br />", $total_price);
      print "Your chosen method of payment is: $payment <br />";
    ?>
  </body>
</html>
```

Notice that the `printf` function is used to implement the numbers that represent money, so exactly two digits appear to the right of the decimal points. Figure 11.6 displays the results of `popcorn3.php`.



Customer:

Joe Popcorn
123 Popcorn Lane
Popcorn City, Iowa, 22222

Order Information

Product	Unit Price	Quantity Ordered	Item Cost
Unpopped Popcorn	$3.00	3	$ 9.00
Caramel Popcorn	$3.50	0	$ 0.00
Caramel Nut Popcorn	$4.50	4	$ 18.00
Toffey Nut Popcorn	$5.00	5	$ 25.00

You ordered 12 popcorn items
Your total bill is: $ 52.00
Your chosen method of payment is: mc

Figure 11.6 The output of `popcorn3.php`

11.11 Files

Because PHP is a server-side technology, it is able to create, read, and write files on the server system. In fact, PHP can deal with files residing on any server system on the Internet, using both HTTP and FTP protocols. We restrict our discussion to dealing with files on the server itself. Furthermore, we will cover only the simple processes of opening, reading, and writing text files. All file operations in PHP are implemented as functions.

11.11.1 Opening and Closing Files

The first step in some file operations is to open it, a process that prepares the file for use and associates a program variable with the file for future reference. This program variable is called the *file variable*. The `fopen` function performs these operations. It takes two parameters: the filename, including the path to it if it is in a different directory, and a use indicator, which specifies the operation or operations that must be performed on the file. Both parameters are given as strings. The `fopen` function returns the reference to the file for the file variable. Every open file has an internal pointer that is used to indicate where the next file operation should take place within the file. We call this pointer the *file pointer*. Table 11.4 describes the possible values of the use indicator.

It is possible for the `fopen` function to fail—for example, if an attempt is made to open a file for reading but no such file exists. It would also fail if the file access permissions did not allow the requested use of the file. The `fopen` function returns `FALSE` if it fails. PHP includes the `die` function, which produces a message and stops the interpretation process. The `die` function is often used

Table 11.4 File use indicators

Use Indicator	Description
`"r"`	Read only. The file pointer is initialized to the beginning of the file.
`"r+"`	Read and write an existing file. The file pointer is initialized to the beginning of the file; if a read operation precedes a write operation, the new data is written just after where the read operation left the file pointer.
`"w"`	Write only. Initializes the file pointer to the beginning of the file; creates the file if it does not exist.
`"w+"`	Read and write. Initializes the file pointer to the beginning of the file; creates the file if it does not exist. Always initializes the file pointer to the beginning of the file before the first write, destroying any existing data.
`"a"`	Write only. If the file exists, initializes the file pointer to the end of the file; if the file does not exist, creates it and initializes the file pointer to its beginning.
`"a+"`	Read and write a file, creating the file if necessary; new data is written to the end of the existing data.

with input and output operations, which sometimes fail. For example, the following statement attempts to open a file named `testdata.dat` for reading only, but calls `die` if the open operation fails:

```
$file_var = fopen("testdata.dat", "r") or
            die ("Error – testdata.dat cannot be opened");
```

This form appears a bit odd, but is exactly what is needed. Because the `or` operator has lower precedence than a function call, `die` will only be called if `fopen` fails, in which case `fopen` returns `FALSE`.

The problem of `fopen` failing because the specified file does not exist can be avoided by determining whether the file exists with `file_exists` before calling `fopen`. The `file_exists` function takes a single parameter, the file's name. It returns `TRUE` if the file exists, `FALSE` otherwise.

A file is closed with the `fclose` function, which takes a file variable as its only parameter.

11.11.2 Reading from a File

The most common way to input a file in PHP is to read its contents into a scalar variable as a string. Then the impressive collection of PHP string manipulation functions can be used to process the file as a string. The `fread` function reads part or all of a file and returns a string of what was read. This function takes two parameters: a file variable and the number of bytes to be read. The reading

operation stops when either the end-of-file marker is read or the specified number of bytes has been read.

Large collections of data are often stored in database systems, so usually only smaller data sets are stored in files. Therefore, files are often read in their entirety with a single call to `fread`. If the whole file is to be read at once, the file's length is given as the second parameter to `fread`. The best way to get the correct file length is with the `filesize` function, so a call to `filesize` is often used as the second parameter to `fread`. The `filesize` function takes a single parameter, the name of the file (not the file variable). For example, to read the entire contents of the file `testdata.dat` as a string into the variable `$file_string`, the following statement could be used:

```
$file_string = fread($file_var,
                     filesize("testdata.dat"));
```

One alternative to `fread` is `file`, which takes a filename as its parameter and returns an array of all of the lines of the file. (A line is a string of non-newline characters, followed by a newline.) One advantage of `file` is that the file open and close operations are not necessary. For example, the following statement places the lines of `testdata.dat` into an array named `@file_lines`:

```
$file_lines = file("testdata.dat");
```

PHP has another file input function that does not require calling `fopen`, `file_get_contents`, which takes the file's name as its parameter. This function reads the entire contents of the file. For example, consider the following call:

```
#file_string = file_get_contents("testdata.dat");
```

A single line of a file can be read with `fgets`, which takes two parameters: the file variable and a limit on the length of the line to be read. Consider the following statement:

```
$line = fgets($file_var, 100);
```

This statement reads characters from `testdata.dat` until it finds a newline character, encounters the end-of-file marker, or has read 99 characters. Note that the maximum number of characters `fgets` reads is one fewer than the limit given as its second parameter.

A single character can be read from a file with `fgetc`, whose only parameter is the file variable. When reading a file by lines or by characters, the read operation must be controlled by the detection of the end of the file. This can be done with the `feof` function, which takes a file variable as its only parameter. It returns a Boolean value: `TRUE` if the last read character of the file was the end-of-file character, `FALSE` otherwise.

11.11.3 Writing to a File

The `fwrite`[8] function takes two parameters: a file variable and the string to be written to the file. It is possible to include a third parameter, which would be used to specify the number of bytes to be written. This parameter is rarely needed. The `fwrite` function returns the number of bytes written. The following is an example of a call to `fwrite`:

```
$bytes_written = fwrite($file_var, $out_data);
```

This statement writes the string value in `$out_data` to the file referenced with `$file_var` and places the number of bytes written in `$bytes_written`. Of course, this will work only if the file has been opened for writing.

The `file_put_contents` function is the counterpart of `file_get_contents`—it writes the value of its second parameter, a string, to the file specified in its first parameter. For example, consider the following call:

```
file_put_contents("savedata.dat", $str);
```

11.11.4 Locking Files

If it is possible for more than one script to access a file at the same time, the potential interference of those accesses can be prevented with a file lock. The lock prevents any other access to the file while the lock is set. Scripts that use such files lock them before accessing them and unlock them when the access is completed. File locking is done in PHP with the `flock` function, which should sound familiar to UNIX programmers. The `flock` function takes two parameters: the file variable of the file and an integer that specifies the particular operation. A value of `1` specifies that the file can be read by others while the lock is set, a value of `2` allows no other access, and a value of `3` unlocks the file.

11.12 Cookies

PHP includes convenient support for creating and using cookies.

11.12.1 Introduction to Cookies

The following introduction to cookies also appears in Chapter 9, "Using Perl for CGI Programming" and in Chapter 10, "Servlets and Java Server Pages." It is repeated here for those who have chosen to skip Chapters 9 and 10.

A *session* is the time span during which a browser interacts with a particular server. A session begins when a browser becomes connected to a particular

8. `fread` has an alias, `fputs`.

server. It ends when the browser ceases to be connected to that server because either it becomes connected to a different server or it is terminated. The HTTP protocol is essentially stateless—it includes no means to store information about a session that is available to a subsequent session. However, there are a number of different reasons why it is useful for the server to be capable of connecting a request made during a session to the other requests made by the same client during that session, as well as previous and subsequent sessions. Many Web sites now create profiles of clients by remembering which parts of the site are perused. Later sessions can use such profiles to target advertising to the client according to the client's past interests. Also, if the server recognizes a request as being from a client who has made an earlier request from the same site, it is possible to present a customized interface to that client. These situations require that information about clients is accumulated and stored.

Cookies provide a general approach to storing information about sessions on the browser system itself. The server is given this information when the browser makes subsequent requests for Web resources from the server. Cookies allow the server to present a customized interface to the client. They also allow the server to connect requests from a particular client to previous requests, thereby connecting sequences of requests into a session.

A *cookie* is a small object of information that consists of a name and a textual value. A cookie is created by some software system on the server. Every HTTP communication between a browser and a server includes a header, which stores information about the message. A message from a browser to a server is a request; a message from a server to a browser is a response. The header part of an HTTP communication can include cookies. So, every request sent from a browser to a server, and every response from a server to a browser, can include one or more cookies.

At the time it is created, a cookie is assigned a lifetime. When the time a cookie has existed reaches its associated lifetime, the cookie is deleted from the browser's host machine. Every browser request includes all of the cookies its host machine has stored that are associated with the Web server to which the request is directed. Only the server that created a cookie can ever receive the cookie from the browser, so a particular cookie is information that is exchanged exclusively between one specific browser and one specific server. Because cookies are stored as text, the browser user can view, alter, or delete them at any time.

Because cookies allow servers to record browser activities, they are considered by some to be privacy concerns. Accordingly, browsers allow the client to change the browser setting to refuse to accept cookies from servers. This is clearly a drawback of using cookies—the clients that reject them render them useless.

11.12.2 PHP Support for Cookies

A cookie is set in PHP with the `setcookie` function. This function takes one or more parameters. The first parameter, which is mandatory, is the cookie's

name given as a string. The second, if present, is the new value for the cookie, also a string. If the value is absent, `setcookie` undefines the cookie. The third parameter, when present, is the expiration time in seconds for the cookie, given as an integer. The default value for the expiration time is zero, which specifies that the cookie is destroyed at the end of the current session. When specified, the expiration time is often given as the number of seconds in the UNIX epoch, which began on January 1, 1970. The `time` function returns the current time in seconds. So, the cookie expiration time is given as the value returned from `time` plus some number. For example, consider the following call to `setcookie`:

```
setcookie("voted", "true", time() + 86400);
```

This call creates a cookie named `"voted"` whose value is `"true"` and whose lifetime is one day (86,400 is the number of seconds in a day).

The `setcookie` function has three more optional parameters, the details of which can found in the PHP manual.

The most important thing to remember about creating a cookie or setting a cookie to a new value is that it must be done before any other XHTML is created by the PHP document. Recall that cookies are stored in the HTTP header of the document returned to the requesting browser. The HTTP header is sent before the body of the document is sent. The server sends the header when it receives the first of the body of the document. So, if any part of the body is created, it is too late to add a cookie to the header. If you create a cookie or change the value of a cookie after even a single character of document body has been generated, the cookie operation will not be successful. (The cookie or the cookie's new value will not be sent to the browser.)

The other cookie operation is getting the cookies and their values from subsequent browser requests. In PHP, cookie values are treated much like form values. All cookies that arrive with a request are placed in the implicit `$_COOKIES` array, which has the cookie names as keys and the cookie values as values. A PHP script can test whether a cookie came with a request by using the `IsSet` predicate function on the associated variable.

As is the case with using cookies with other technologies, remember that cookies cannot be depended upon because some users set their browsers to reject all cookies. Furthermore, most browsers have a limit on the number of cookies that will be accepted from a particular server site.

11.13 Session Tracking

In many cases, information about a session is needed only during the session. Also, the needed information about a client is nothing more than a unique identifier for the session, which is commonly used in shopping cart applications. For these cases, a different process, named *session tracking*, can be used. Rather than using one or more cookies, a single session array can be used to store information about the previous requests of a client during a session. In particular, ses-

sion arrays often store a unique session ID for a session. One signficant way that session arrays differ from cookies is that they can be stored on the server, whereas cookies are stored on the client.

In PHP, a session ID is an internal value that identifies a session. Session IDs need not be known or handled in any way by PHP scripts. PHP is made aware that a script is interested in session tracking by calling the `session_start` function, which takes no parameters. The first call to `session_start` in a session causes a session ID to be created and recorded. On subsequent calls to `session_start` in the same session, the function retrieves the `$_SESSION` array, which stores any session variables and their values that were registered in previously executed scripts in this session.

Session key/value pairs are created or changed by assignments to the `$_SESSION` array. They can be destroyed with the `unset` operator. Consider the following example:

```
session_start();
if (!IsSet($_SESSION["page_number"]))
  $_SESSION["page_number"] = 1;
$page_num = $_SESSION["page_number"];
print("You have now visited $page_num page(s) <br />");
$_SESSION["page_number"]++;
```

If this is not the first document visited that calls `session_start` and sets the `page_number` session variable, this script will produce the specified line with the last set value of `$_SESSION["page_number"]`. If no document that was previously visited in this session set `page_number`, this script sets `page_number` to `1`, produces the following line, and increments `page_number`:

```
You have now visited 1 page(s)
```

Summary

PHP is a server-side, XHTML-embedded scripting language. The language is similar to both Perl and JavaScript. The PHP processor takes as input a file of XHTML/PHP, copies the XHTML to an output file, and interprets the PHP script in the input file. The output of any PHP script is written into the output file. PHP scripts are either directly embedded in XHTML files or are referenced in the XHTML files and subsequently copied into them.

PHP has four scalar types: integer, Boolean, double, and string. PHP variable names all begin with dollar signs. The language is dynamically typed. Arithmetic and Boolean expressions in PHP are very similar to those in other common languages. PHP includes a large number of functions for arithmetic and string operations. The current type of a variable is maintained internally and can be determined by a script through several different built-in functions.

The `print` and `printf` functions are used to produce output, which becomes part of the PHP processor output file. The control statements of PHP are similar to those of other common programming languages.

PHP's arrays are a combination of the traditional arrays of C and its descendant languages and hashes. Arrays can be created by assigning values to their elements. This is often done with the `array` construct, which allows the specification of values and optionally the keys for one or more elements of an array. PHP has predefined functions for many array operations. Among these are `explode` and `implode` for converting between strings and arrays; `current`, `next`, and `prev` for fetching elements in sequential order; `each` for obtaining both the keys and values of the elements of an array in sequential order; and `array_keys` and `array_values`, which return an array of the keys and values of the array, respectively. There are also functions for stack operations on arrays. The `foreach` statement, similar to that of Perl, provides sequential access to the elements of an array. Finally, PHP has a collection of functions for sorting the elements of arrays in various ways.

User-defined functions in PHP are similar to those of other languages, except for parameter passing. Because PHP does not have pointers, pass-by-reference parameters must be specified in either the function call or the function definition. Variables used only in a function are local to that function. Access to variables used outside a function is specified with a `global` declaration. Static variables can be declared with a `static` declaration.

PHP's pattern matching can use either POSIX-style or Perl-style regular expressions. Form data is placed in user-accessible variables implicitly by the PHP system. This makes form handling very convenient.

Files are opened and prepared for reading, writing, or both with the `fopen` function, which returns a file variable. Every file has a file pointer, which maintains a position in the file where the next read or write will take place. Files can be read with `fread`, which reads as many bytes as specified in the call, up to the whole file, into a string. The lines of a file can be read into an array of strings with `file`. A single character of a file can be read with `fgetc`. The `file_get_contents` function is used to read an entire file into a variable, without needing the `fopen` function. The `fwrite` function is used to write to a file. The `file_put_contents` function writes a string to a file, without needing the `fopen` function. Interference among simultaneous file accesses can be avoided by locking a file with `flock`.

Cookies are created and set to values with the `setcookie` function, which has parameters for the cookie name, its value, and a lifetime in seconds. Cookies created or set in a previous script are available to a current script directly through the `$_COOKIES` array. A script can test whether a cookie exists and is set to a value with `IsSet`. Session tracking is relatively simple in PHP. The `session_start` function creates a session ID. Session variables are stored in the `$_SESSION` array.

Review Questions

11.1 How does a Web server determine whether a requested document includes PHP code?

11.2 What are the two modes of the PHP processor?

11.3 What are the syntax and semantics of the `include` construct?

11.4 Which parts of PHP are case sensitive and which are not?

11.5 What are the four scalar types of PHP?

11.6 How can a variable be tested to determine whether it is bound?

11.7 How can you specify to the PHP processor that you want uses of unbound variables to be reported?

11.8 How many bytes are used to store a character in PHP?

11.9 What are the differences between single- and double-quoted literal strings?

11.10 If an integer expression appears in Boolean context, how is its Boolean value determined?

11.11 What happens when an integer arithmetic operation results in a value that cannot be represented as an integer?

11.12 If a variable stores a string, how can the character at a specific position in that string be referenced?

11.13 What does the `chop` function do?

11.14 What is a coercion?

11.15 What are the three ways the value of a variable can be explicitly converted to a specific type?

11.16 How can the type of a variable be determined?

11.17 If a string is compared with a number, what happens?

11.18 What is the advantage of using the unique closing reserved words such as `endwhile`?

11.19 In what two ways can arrays in PHP be created?

11.20 What keys are used when an array is created but no keys are specified?

11.21 Must all of the values of an array be of the same type?

11.22 Must all of the keys of an array be of the same type?

11.23 What exactly do the `array_keys` and `array_values` functions do?

11.24 What exactly does the `in_array` function do?

11.25 Explain the actions of the `implode` and `explode` functions.

11.26 Describe the actions of the `next`, `reset`, and `prev` functions.

11.27 What are the syntax and semantics of the two forms of the `foreach` statement?

11.28 Describe the result of using the `sort` function on an array that has both string and numeric values.

11.29 What is the difference betweeen the `sort` and `asort` functions?

11.30 What happens if a script defines the same function more than once?

11.31 Are function names case sensitive?

11.32 What value is returned by a function if its execution does not end by executing a `return` statement?

11.33 What are the two ways you can specify that a parameter is to be passed by reference?

11.34 How can a variable used outside a function be accessed by the function?

11.35 How can you define a variable in a function so that its lifetime extends beyond the time the function is in its first execution?

11.36 How can the value of a form element be accessed by a PHP script?

11.37 What is a file variable?

11.38 What is a file pointer?

11.39 What does an `fopen` function return if it fails?

11.40 Explain the parameters and actions of the `fread` function.

11.41 What is returned by the `fwrite` function?

11.42 How can a cookie be created in a PHP script?

11.43 How can a script determine whether a particular cookie exists?

11.44 How can a variable be saved in a session?

Exercises

Write, test, and debug (if necessary) PHP scripts for the following specifications. For Exercises 11.1 to 11.7, write functions and the code to test them.

11.1 *Parameter*: An array of strings.

Return value: A list of the unique strings in the parameter array.

11.2 *Parameter*: An array of numbers.

Return value: The average and median of the parameter array.

11.3 *Parameter*: An array of strings.

Return value: A list of the three strings that occur most frequently in the parameter array.

11.4 *Parameters*: An array of numbers (pass-by-value) and two arrays (pass-by-reference).

Return value: None.

Result: The first pass-by-reference parameter must have the values of the given array that are greater than zero; the second must have the values that are less than zero.

11.5 *Parameter*: A string of numbers separated by spaces.

Return value: The first four-digit number in the string; false if none.

11.6 *Parameter*: A file variable of a file of text, where the words are separated by spaces or colons.

Return value: The word that appears most often in the file.

11.7 *Parameter*: A string containing words that are delimited on the left with spaces and on the right with spaces, commas, periods, or question marks.

Return value: The three most common words in the string that have more than three letters.

11.8 Modify the sample script in Section 11.9, `word_table.php`, to place the output table in an XHTML table.

11.9 Write an XHTML document to that includes an anchor tag that calls a PHP document. Also write the called PHP document, which returns a randomly chosen greeting from a list of five different greetings. The greetings must be stored as constant strings in the script. A random number between 0 and 4 can be computed with these lines:

```
# Set the seed for mtrand with the number of microseconds
#  since the last full second of the clock
mt_srand((double)microtime() * 1000000);
$number = mtrand(0, 4);  # Computes a random integer 0-4
```

11.10 Write the XHTML code to create a form with the following capabilities:

a. A text widget to collect the user's name

b. Four checkboxes, one each for the following items:

i. Four 100-watt light bulbs for $2.39

ii. Eight 100-watt light bulbs for $4.29

iii. Four 100-watt long-life light bulbs for $3.95

iv. Eight 100-watt long-life light bulbs for $7.49

c. A collection of three radio buttons that are labeled as follows:
   i. Visa
   ii. MasterCard
   iii. Discover

11.11 Write a PHP script that computes the total cost of the ordered light bulbs from Exercise 11.10 after adding 6.2 percent sales tax. The program must inform the buyer of exactly what was ordered, in a table.

11.12 Write the XHTML code to create a form that collects favorite popular songs, including the name of the song, the composer, and the performing artist or group. This document must call one PHP script when the form is submitted and another to request a current list of survey results.

11.13 Write the PHP script that collects the data from the form of Exercise 11.12 and writes it to a file.

11.14 Write the PHP script that produces the current results of the survey of Exercise 11.12.

11.15 Write the XHTML code to provide a form that collects names and telephone numbers. The phone numbers must be in the format ddd-ddd-dddd. Write a PHP script that checks the submitted telephone number to be sure that it conforms to the required format and then returns a response that indicates whether the number was correct.

11.16 Modify the PHP script for Exercise 11.9 to count the number of visitors and display that number for each visitor. *Hint*: Use a file to store the current count.

CHAPTER

12

Introduction to ASP.NET

12.1 Overview of the .NET Framework
12.2 Introduction to C#
12.3 Introduction to ASP.NET
12.4 ASP.NET Controls
12.5 Web Services
Summary • Review Questions • Exercises

This chapter introduces ASP.NET and discusses its use for developing Web applications on Microsoft's .NET computing platform. Before describing ASP.NET, it is necessary to briefly introduce the .NET Framework, of which it is a part, and the programming language used in this chapter to discuss ASP.NET, C#. After these preliminaries, ASP.NET is introduced, including the structure of ASP.NET documents and code-behind files. Next, HTML controls are introduced. To describe the processing of ASP.NET pages, the whole life cycle of that processing is briefly discussed. Then page-level and control events are covered. Following this, a different category of controls is introduced, the Web controls. Among the Web controls is a collection of controls used to validate form data, the validation controls. The last section of the chapter introduces Web services using ASP.NET. Six complete examples are used to illustrate the concepts discussed.

The reader must keep in mind that many books have been devoted to describing ASP.NET. So, this one chapter can provide just a brief overview of

this complex and powerful technology. Also, the chapter uses only about nine pages to introduce C#. If the reader is not familiar with Java, this is wholly inadequate. Such readers are advised to study Appendix A before tackling this chapter. However, because of the similarity of C# to Java, Java programmers should be able to use C# for ASP.NET documents after studying this chapter.

12.1 Overview of the .NET Framework

.NET is an umbrella term for a collection of technologies that was announced by Microsoft in early 2000. In January 2002, the software to support .NET was released. It was quickly adopted by a significant part of the Web software industry. It will undoubtedly continue to be a major player in this industry in the future.

12.1.1 Background

.NET was developed in recognition that the future of a significant part of the computing business lies in Web-based software services, in which components of a software system may reside on different computers in different places on the Internet. Prior to .NET, Microsoft's technology for distributed component-based systems was named COM.

A *component* is an encapsulation of software that can stand by itself and be used by other components, without those components knowing how the functionality of the component was implemented. Components can also be created with technologies other than COM. JavaBeans is a technology developed by Sun Microsystems to support distributed component-based computing using Java. The primary difference between JavaBeans and COM components is that COM components can be written in a variety of different programming languages—they are language neutral.

The .NET Framework is exactly that—a framework for the development and deployment of .NET software. In .NET, the central concept is that a software system or service consists of a collection of components that can be written in different languages and reside on different computers in different locations. Also, because of the diversity of employed languages, the collection of tools for development and deployment must be language neutral. These ideas permeate all of the parts of the .NET Framework.

12.1.2 .NET Languages

Initially, .NET included five languages: Visual Basic .NET (VB .NET), Managed C++ .NET, JScript .NET, J# .NET, and a new language, C#. VB .NET is based on VB 6.0, a language widely used for Web programming and other software development that includes graphical user interfaces (GUIs). VB .NET differs from VB in many ways, most importantly in that it is a full-fledged object-oriented language, whereas VB is not. Managed C++ .NET is a garbage-

collected version of C++. JScript .NET is based on JavaScript but also provides full support for object-oriented programming. J# .NET is Microsoft's version of Java. C# is briefly described in Section 12.2. There are now more than 20 available languages that run under .NET, among which are COBOL, Eiffel, Fortran, Perl, and Python. Work is underway to add more languages to the list.

The multilanguage aspect of .NET sets it apart from other such systems. The advantage of supporting a variety of programming languages is that there is an easy migration path from software in many different languages to .NET. Organizations that make heavy use of any of the .NET languages can easily transition to .NET. Programmers who are experienced and skilled in almost any common language can quickly become productive in a .NET environment. Although it makes reuse much more feasible, having a system composed of components written in different languages is not all good. One important disadvantage is that it complicates maintenance.

12.1.3 The Common Language Runtime

The base technology for .NET is the Common Language Runtime (CLR), which provides language-neutral services for the processing and execution of .NET software. Among the most important services of the CLR are garbage collection, type checking, debugging, and exception handling. These services are used for all of the .NET languages.

For every .NET language, the CLR has a compiler to translate source programs to a common intermediate language, which was originally named Microsoft Intermediate Language (MSIL) but now is usually called Intermediate Language (IL). After compilation, all IL programs have the same form, regardless of the original source language. Before execution, IL programs are incrementally compiled to machine code for the host machine by a Just-In-Time (JIT) compiler, which is part of the CLR. A JIT compiler translates a method to machine code only when the method is called. Once compiled, the machine code version of the method is kept for the duration of execution of the program so that subsequent calls do not require recompilation. Because many executions of a program do not cause all of the program's methods to be called, this is an efficient approach to compilation. In .NET, it is also possible to compile a whole program into machine code before execution begins. JIT compilers are commonly used for Java program execution. One major difference between Java's approach to program execution and that of the .NET languages is that IL programs are never interpreted, as bytecode (the Java intermediate language) programs sometimes are. In fact, the .NET Framework does not include an IL interpreter, which would be similar to the Java Virtual Machine.

12.1.4 The Common Language Infrastructure

To make it possible to use the CLR for multiple languages, those languages must adhere to a set of common characteristics. These are specified by the Common

Language Infrastructure (CLI), which consists of two specifications, the Common Type System (CTS) and the Common Language Specification (CLS).

The CTS defines a set of types that are supported by .NET languages. It also provides a mapping from every type in each language to its corresponding common type. For example, the CTS defines a type named `Int32`, which is a 32-bit signed integer type. The C# type `int` corresponds to `Int32`. The concept of common base types is analagous to what is done with CORBA (`http://www.corba.org`), which defines a similar set of types and gives a mapping from various languages to these common types. In CTS, types occur in two natural categories, value types and reference types. *Value types* directly reference values in memory cells; that is, the value of a value type object is a value. *Reference types* refer to or address a memory cell that has a value. So, the value of a reference type is not a value; it is an address.

Having common types among languages is, of course, necessary if components in those languages are expected to interoperate correctly. All types of all .NET languages derive from a single type, `System.object`.

The CLS defines the language features that must be supported by all .NET languages. .NET languages can, however, include features beyond what is specified in CLS. Of course, use of such features in a program will jeopardize the possibility of interoperation of that program with programs in languages that do not support those features. Examples of CLS restrictions follow:

1. No operator overloading
2. No pointers
3. Identifiers are not case sensitive

Interestingly, the new .NET language, C#, includes all of these. However, they should not be used in C# programs that will interoperate with components written in other .NET languages that do not include them. For example, VB .NET identifiers are not case sensitive. If a C# component must interoperate with a VB .NET component, the C# component must not use two different identifiers in the interface to the VB .NET component whose only difference is case (for example, `Sum` and `sum`). To design a language that can be a .NET language, the designer must ensure that all of the CLI features are supported.

The .NET Framework includes a large collection of class libraries called the Framework Class Libraries (FCL). The initial release of FCL included more than 4,000 classes that support a wide array of application areas. For example, there are APIs for networking, reflection, Web forms, database access, and file system access. Also included are APIs for access to Windows features such as the registry, as well as other Win32 functions. These functions are called through FCL classes and are executed in the CLR.

Perhaps the most important result of having the CLI and the CLR is that components written in any of the .NET languages can use any class in the FCL. More striking, perhaps, is the result that a component in any .NET language can use classes defined in any other component written in any other .NET lan-

guage. This enables a program to call the methods of a class written in any other .NET language. It also allows subclassing classes written in any other .NET language. For example, a C# program can subclass a class written in VB .NET. It can also call the methods of a class written in managed C++.

12.2 Introduction to C#

This section provides a brief introduction to C#. It is written with the assumption that the reader is familiar with Java. C# is used for the examples in this chapter, but little of the language used will be unfamiliar to Java programmers.

12.2.1 Origins

C# is a new object-oriented language, designed to fit the needs of .NET programming. Like most other "new" programming languages, most of C# is not in fact new but borrowed from existing languages. C# can be thought of as the most recent iteration of the sequence of C-based languages. C++ was derived from C (and SIMULA 67), and Java was derived, at least partially, from C++. C# is derived from both C++ and Java, having been based on Java but including a number of features that are part of C++ but not Java. From Java, C# gets single inheritance, interfaces, garbage collection, the absence of global types or variables, and its level of assignment type coercion. From C++, C# gets pointers, operator overloading, a preprocessor, structs, and enumerations (although its structs and enumerations differ significantly from those of C++). From Delphi and VB, C# inherits properties. Finally, from J++ (Microsoft's version of Java), C# gets delegates. Among the C# features that are new are indexes, attributes, and events. Overall, C# is less complex than C++ without giving up much of the expressivity of C++, which is also the case with Java. Although C# is more complex than Java, it is also more powerful.

12.2.2 Primitive Types and Expressions

C# has two categories of data: primitives and objects. C# includes a long list of primitive types, ranging from `byte`, which is an unsigned one-byte integer, and `char`, which is a two-byte Unicode character, to `int`, `float`, `double`, and `decimal`, which is a 16-byte decimal type that can store up to 28 decimal digits. All of the primitive types of C# are briefly described in Table 12.1.

C# supports pointers, as in C++, although they are rarely used. They were included in the language to allow interoperability with C and C++ modules that use pointers. Any method that uses a pointer must be marked with the reserved word `unsafe` in its header.

C# has one nonprimitive value type, enumerations, which is discussed in Section 12.2.3.

Table 12.1 The primitive types of C#

Type	Description
`byte`	Unsigned integer (1 byte)
`char`	Unicode character (2 bytes)
`bool`	Boolean (1 byte)
`sbyte`	Signed integer (1 byte)
`short`	Signed integer (2 bytes)
`ushort`	Unsigned integer (2 bytes)
`int`	Signed integer (4 bytes)
`uint`	Unsigned integer (4 bytes)
`float`	Floating point (4 bytes)
`double`	Double precision floating point (8 bytes)
`decimal`	Fixed-precision up to 28 decimal digits
`long`	Signed integer (8 bytes)
`ulong`	Unsigned integer (8 bytes)

Symbolic constants are defined by preceding the type name in a declaration with the `const` reserved word. Every symbolic constant declaration must include an initial value. For example:

```
const float pi = 3.14159265;
```

C# has the same collection of arithmetic operators as Java, so its expressions are also like those of Java. Likewise, the C# assignment statements are identical to those of Java.

C# has 77 reserved words (PHP has 29, Java has 48), so the beginning C# programmer should keep that list handy when writing code.

12.2.3 Data Structures

The .NET FCL defines an extensive variety of collection classes, including `Array`, `ArrayList` (dynamic length arrays), `Queue`, and `Stack`. Although `Array` is a class, the syntax of array references is exactly like that of C. Because it is a class, array access is through reference variables. The following is an example of a declaration of a reference to an `int` array:

```
int[] myIntArray;
```

The variable `myIntArray` can reference any single-dimensioned array of `int` elements. An array object is created with the `new` operator, as in the following statement:

```
myIntArray = new int[100];
```

`myIntArray` now references an array of 100 integers on the heap.

The `Array` class provides a large collection of methods and properties. Among the methods are `BinarySearch`, `Copy`, and `Sort`. The most frequently used property is `Length`. For example, the following assignment statement sets `len` to `100`:

```
len = myIntArray.Length;
```

C# includes an enumeration type, which is a value type. In an enumeration type, the type's designer lists all of the possible values of the type in its declaration. Internally, enumeration values are stored as integers. In C and C++, enumeration type variables can be treated like integers, meaning that they can be operated on with arithmetic operators, which for many enumeration types is nonsense. Also, it is possible to assign an integer value to an enumeration type that is not one used in the internal representation of the enumeration type. Syntactically, C#'s enumerations look like those of C and C++ but are actually much safer. The increased security results from the fact that enumeration types are never coerced to other types, and other types are never coerced to enumeration types. This prevents enumeration variables from having values that are not defined for the enumeration type. It also prevents arithmetic operators from being used on enumeration values. The following is an example of the definition of an enumeration type in C#:

```
enum Colors {blue, green, yellow, red, orange, purple};
```

12.2.4 Control Statements

The control statements of C# are nearly identical to those of Java (as well as the other C-based languages). Two differences are the `foreach` and `switch` statements. The `foreach` statement is a data-structure-controlled iterator. It can be used on arrays and other collections. The syntax of `foreach` is as follows:

`foreach (`*type identifier* `in` *collection*`) { ... }`

For example:

```
foreach (int myInt in myIntArray) { ... }
```

The `switch` statement of C# is similar to that of Java but with one important restriction. The `switch` statements of C, C++, and Java all suffer the same problem: Although in the vast majority of cases, control should exit the construct after a selected segment has executed, the default is that control flows to

the next segment after the selected segment has executed. Therefore, most segments in `switch` constructs must include a `break` statement. Leaving out the `break` is a common error in `switch` constructs. To avoid these errors, the C# `switch` has the requirement that every selectable segment in a `switch` construct must end with an unconditional branch instruction, either a `break` or a `goto`. To force control to continue to the next segment, a `goto` is used. For example, consider the following `switch` construct:

```
switch (value) {
   case -1:
      Negatives++;
      break;
   case 0:
      Zeros++;
      goto case 1;
   case 1:
      Positives++;
   default:
      Console.WriteLine("Error in switch \n");
}
```

Note that `WriteLine` is a method of the `Console` class that is used to produce output to the screen.

12.2.5 Classes, Methods, and Structures

C# is a pure object-oriented programming language in the same sense as Java. There are no subprograms except methods, which can only be defined in classes (and structs) and can only be called through objects or classes. Most of the syntax and semantics of C# classes and methods are the same as those of Java. In the following paragraphs, the most important differences are discussed.

Parameters to methods can be passed by value, passed by reference, or passed by result. These three implement in mode, which is the default mode (one-way communication to the method), inout mode (two-way communication between the caller and the called method), and out mode (one-way communication from the called method to the calling method) parameter semantics, respectively. Reference variables implicitly have pass-by-reference semantics. Pass by reference is specified for value types by preceding the formal parameter with the `ref` reserved word. Pass by result is specified for value types by preceding the formal parameter with the `out` reserved word.

In some object-oriented languages, such as Java, it is relatively easy to write methods that accidentally override inherited methods.[1] This happens because the author of the new method either forgets or is unaware that a method with

1. This would only happen if the author of the inherited method wants to allow it to be overridden somewhere among the class descendants. If the method should never be overridden, it is marked `final`, which prevents all descendant classes from overriding it.

the same name already exists in the class ancestry. To avoid this error, C# requires any method that is allowed to be overriden to be marked `virtual`. Furthermore, any method that is meant to override an inherited method must be marked `override`. If a method is defined that has the same protocol as an inherited method but is not meant to override it, it must be marked `new`. Such a method hides the inherited version.

A C# method can take a variable number of parameters as long as all are of the same type. The method defines just one parameter, an array with the `params` qualifier. For example:

```
void SumInts(params int [] intValues) { ... }
```

This method can be called with an array or a list of integer expressions. For example:

```
int [] myIntArray = new int[6] {2, 4, 6, 8, 10, 12};
sum1 = SumInts(myIntArray);
sum2 = SumInts(10, i, 17, k);
```

A struct in C++ is very similar to a class. In C#, however, a struct is quite different from the classes of the language. A C# struct is a lightweight class that does not support inheritance or subclassing. However, C# structs can implement interfaces and have constructors. Structs are value types, which means they are allocated on the runtime stack. The syntactic form of a struct declaration is identical to that of a class, except the reserved word `struct` is used in place of `class`. All C# primitive types are implemented as structs.

12.2.6 Properties

One problem in object-oriented programming languages is the conflict between the theoretical goal of having all data in classes be hidden from code outside the class (by defining it to be private) and the occasional need for other classes to access or modify such data. In many languages, the solution is to define accessor methods that provide indirect access to private data fields. C# borrows the Delphi and VB solution, properties. A *property* is a special data field of a class that can provide get and set accessors. Neither of these needs to be explicitly called—they are implicitly called when code accesses or modifies the property's data. Consider the following example of a property named `DegreeDays`, which defines a private data field, `degreeDays`, and provides indirect access to it:

```
public class Weather {
  public int DegreeDays {
    get {
      return degreeDays;
    }
    set {
      degreeDays = value;
    }
```

```
    private int degreeDays;
  }  //** end of property DegreeDays
  ...
}  //** end of class Weather
...
Weather w = new Weather();
...
w.degreeDays += degreeDaysToday;
```

In the last assignment statement of this code, the data field of the `DegreeDays` property, `degreeDays`, is accessed implicitly through the `get` method. It is also modified, implicitly through the `set` method, in the same statement. In effect, the property allows access to a private date field. Note the use of the implicit variable `value` in the `set` method, which references the value of the property's data. One advantage of a property over simply making the field public is that read-only access can be provided (by defining a `get` method but not a `set` method). Furthermore, the `set` method can be defined to restrict the range of values that can be assigned to the data field.

12.2.7 Delegates

Delegates are object-oriented method pointers. They are reference types for methods of specific protocols. Delegates are created with declarations such as the following:

```
public delegate void AHandler(
                     object o, System.EventArgs e);
```

The `AHandler` delegate can reference any method that returns `void` and takes parameters of `object` and `System.EventArgs` types. (This is the protocol for event handlers in .NET.) This delegate encapsulates all such methods that are subscribed to it. The delegate maintains a list of the methods that have been subscribed to it. Subscribing a method to a delegate is similar to the process of registering an object to be an event listener for a particular event in Java. The subscribed methods can be called through the delegate. In Section 12.4.4, delegates will be used for event handling of form widget events. The subscription process will be described there.

12.2.8 Program Structure

In contemporary object-oriented programming languages, programs have access to large, comprehensive, and complex class libraries that provide services and commonly needed types. For .NET, this is the FCL. The most commonly used part of the .NET FCL is `System`. This class defines a namespace for its constituents, also named `System`. The `System` namespace provides classes for input and output, string manipulation, event handling, threading, and collections, among others. One of the input and output classes of `System`, `Console`,

supports console applications, which are those that are not interactive. Console applications are run from a command prompt rather than through a graphical user interface. The `Console` class has methods `ReadLine` and `WriteLine` to read strings from the keyboard and write strings to the screen, respectively.

The `using` statement is used to abbreviate the names of classes in a namespace. For example,

```
using System;
```

allows the program to access the classes defined in `System` without using the prefix `System` on the names of those classes. For example, if `using System` is not included in a program, `WriteLine` must be called with `System.Console.WriteLine`. However, if `using System` is included, then `WriteLine` can be called with `Console.WriteLine`.

The main method of every C# program is named `Main` (not `main`). `Main` does not need to have a parameter, and it can return either `int` or `void`.

An example of a minimal but complete program follows:

```
using System;
public class myTester {
  public static void Main() {
    Console.WriteLine("Howdy!");
  }
}
```

12.2.9 File Storage for Programs

Any number of public classes and/or interfaces can be stored in one file.[2] Every class in a file can define a `Main` method. If the file has more than one `Main` method, the compiler must be told which one should be called first. This is done on the compile command-line command, `csc`. For example, if the file `myClassFile.cs` has more than one class with a `Main` method, and the one that is to be executed first is in the class `myMainClass`, the file must be compiled with the following:

```
csc myClassFile.cs /main:myMainClass
```

Unlike Java, C# class names and the files where the classes are stored do not need to be related. Names of files for class definitions should use the `.cs` extension, but that is not required.

Most C# programs, and most ASP.NET documents for that matter, are written using Microsoft's Visual Studio, a powerful development environment. Because of the complexity of this system, it is not described in this chapter. Both C# programs and simple ASP.NET documents can be written without Visual Studio. In fact, all of the code in this chapter was written without Visual Studio. This makes the examples cleaner and simpler to discuss and understand.

---

2. This differs from Java, in which a file can contain only one public class.

Following is the listing of a file that contains two class definitions, one to define a stack implemented in an array and one to test the stack class. It is meant to illustrate the basics of classes and methods in C#.

```
// stackClass.cs - a file containing two class definitions
//  1. A class, StackClass, that implements a stack in an
//     array of length 100
//  2. A class, TstStack, to test the StackClass

using System;

public class StackClass {
  private int [] stackRef;
  private int maxLen,
              topIndex;

// A constructor for StackClass
  public StackClass() {
    stackRef = new int [100];
    maxLen = 99;
    topIndex = -1;
  }

// The push method
  public void push(int number) {
    if (topIndex == maxLen)
      Console.WriteLine("Error in push—stack is full");
    else stackRef[++topIndex] = number;
  }

// The pop method
  public void pop() {
    if (topIndex == -1)
      Console.WriteLine("Error in pop—stack is empty");
    else --topIndex;
  }

// The top method
  public int top() {return (stackRef[topIndex]);}

// The empty predicate method
  public bool empty() {return (topIndex == -1);}
}

// A class to test StackClass
```

```
public class TstStack {
  public static void Main() {

// Create an instance of StackClass
    StackClass myStack = new StackClass();

// Test push, top, pop, and empty
    myStack.push(42);
    myStack.push(29);
    Console.WriteLine("29 is: " + myStack.top());
    myStack.pop();
    Console.WriteLine("42 is: " + myStack.top());
    myStack.pop();
    myStack.pop();  // Produces an error message
  }
}
```

12.3 Introduction to ASP.NET

ASP.NET is a large and complex topic. This section provides a brief introduction to the fundamentals of ASP.NET.

12.3.1 The Basics

ASP.NET is a Microsoft technology for building dynamic Web documents (ASP is an acronym for Active Server Pages). Dynamic ASP.NET documents are supported by programming code executed on the Web server. Although ASP.NET documents can also include client-side scripts, we focus on the server side. ASP.NET is based on its predecessor, ASP, which allowed embedded scripts written in either JScript (Microsoft's JavaScript) or VBScript, a scripting dialect of VB. Both of these languages are purely interpreted, making their execution much slower than that of code written in compiled languages. There are a few problems with using purely interpreted code to provide server-side dynamic documents. First, documents that include both scripting code and XHTML are complex, especially if they are large. Mixing markup and programming code, which mixes presentation and business logic, creates a confusing document. Furthermore, Web markup designers and programmers must deal with the same document. Second, purely interpreting scripts before delivering documents is inefficient. Third, there is the problem of reliability of code written in scripting languages, in part because they use either dynamic typing or relaxed typing rules. Also, in some scripting languages array index ranges are not checked.

As we saw in Chapter 10, "Servlets and Java Server Pages," JSP offers one solution to these problems: Use Java to describe the computation associated

with user interactions with Web documents. The Java language is much more reliable than the scripting languages, largely because of the strict type checking and array index range checking. Furthermore, compiled Java code is faster than interpreted scripting code. Finally, although Java can be directly embedded in XHTML documents with JSP, it is entirely separate when JavaBeans is used.

ASP.NET provides an alternative to JSP, with two major differences: First, ASP.NET allows the server-side programming code to be written in any of the .NET languages. Second, in ASP.NET all programming code is compiled, which allows it to execute much faster than interpreted code.

Programming code that is part of a Web application but resides outside of the ASP.NET document (the XHTML document file) is placed in a file called the *code-behind* file. This is a good approach to adding code to a document to produce a dynamic document because it clearly separates the static XHTML from the program code. This separation cleanly reflects the difference between the kinds of people who design these two kinds of artifacts. It also simplifies both the ASP.NET document and the code file.

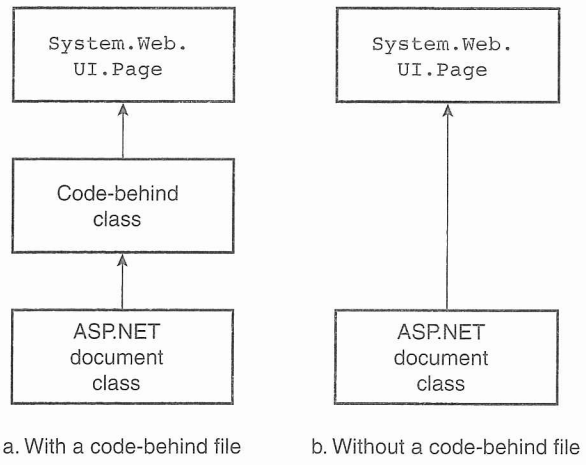
Every ASP.NET document is compiled into a class, which resides in an assembly. An assembly is the unit in which compiled classes are stored in .NET. Compiling a markup document, which may or may not include embedded programming code, into a class is an abrupt departure from other approaches to developing Web resources. From a programmer's point of view, developing dynamic Web documents (and the supporting code) in ASP.NET is similar to developing non-Web applications. Both involve defining classes based on library classes, implementing interfaces from a library, and calling methods defined in library classes. An application class uses and interacts with existing classes. In ASP.NET, this is exactly the same for Web applications. Web documents are designed by designing classes.

The class to which an ASP.NET document is compiled is a descendant of `System.Web.UI.Page`, from which it inherits a collection of members. Among the most commonly used of these are the `Request` and `Response` objects, the `HTMLControls` and `WebControls` classes, and the `IsPostBack` property. The `Write` method of the `Response` object is used to create output from an ASP.NET document. The two controls classes define the large collection of server-side widgets that are available to ASP.NET documents. Sample documents that use the controls classes appear in Section 12.4. The `IsPostBack` property is used to determine whether the current request is a result of a user interaction with a form (as opposed to an initial request for a document). Its use is illustrated in a sample document in Section 12.4.2.

ASP.NET documents that do not use code-behind files are compiled into direct subclasses of `Page`. Code-behind files also are compiled into subclasses of `System.Web.UI.Page`. We call the class that results from compiling the ASP.NET document the *document class*. Note that a document class is pure C# source code rather than an intermediate code version.[3] Document classes that

3. This is technically a misuse of the term "compile," although ASP.NET documents are translated from their original form to C# programs.

use a code-behind file are subclasses of the code-behind class. The code-behind class is an intermediate class between the document class and `System.Web.UI.Page`. So, programming code in an ASP.NET document inherits from both `Page` and the class of the code-behind file. Inheritance diagrams for ASP.NET documents with and without code-behind files are shown in Figure 12.1.



a. With a code-behind file b. Without a code-behind file

Figure 12.1 Inheritance diagrams for ASP.NET documents with and without code-behind files

12.3.2 ASP.NET Documents

ASP.NET documents can include a number of different elements. First, they can contain XHTML markup, including standard XHTML comments. This markup can include static XHTML, XHTML elements made dynamic by client-side scripts, and XHTML elements made dynamic by server-side code. Second, an ASP.NET document can include one or more directives, the most common of which is `@Page`, which can have any of a large number of different attributes. Third, documents can have *render blocks*, which use the `<%` opening tag and `%>` closing tag and have programming code as content. This code, which cannot include subprogram definitions, is placed (by the document compiler) in a function of the class of the document when the document is translated to a class. This function's body typically consists of method calls and output statements to create the static XHTML of the document. The function is executed when the document class is executed (which produces the XHTML document that is returned to the requesting browser). Fourth, documents can include programming code as the content of script elements that include the attribute `runat` set to `"server"`. Such code is called a *declaration block* because it is not implicitly executed. Subprograms, including event handlers, are defined

in declaration blocks. The code in declaration blocks is inserted directly into the class created for the document. Finally, documents can include server-side comments, which appear in elements that use the opening tag `<%--` and the closing tag `--%>`.

Directives appear in the same tag form as render blocks. Their names begin with at signs (`@`), although in a document the `@` is not usually attached to the name. For example, the general form of a directive is as follows:

```
<%@ directive-name attributes %>
```

The only directive required in every ASP.NET document that includes embedded programming code is `@Page`. For these documents, the `@Page` directive must minimally include a `Language` attribute, which is assigned the name of the .NET language that is used for embedded programming code in the document. This, of course, is necessary to inform the CLR which compiler is to be used to compile the document.

Output from an ASP.NET document, which becomes part of the XHTML document returned to the browser, is generated through a `Response` object. The `Response` class defines the `Write` method, whose string parameter is markup code. For example, consider the following:

```
Response.Write("<h1> Today's Report </h1>");
```

If the output must be formatted, for example, to include the values of variables, the `Format` method of the `string` type (`string` is an alias for the `System.String` class) is used. The parameter to `Format` has the same form as the parameter to `Console.WriteLine`. For example, consider the following:

```
string msg = string.Format("The answer is: {0} <br />",
                           answer);
Response.Write(msg);
```

At this point, an example is in order. The following simple ASP.NET document uses C# code in a render block to call a subprogram, defined in a declaration block, to create an array of ten random numbers. The render block code then displays the array.

This example uses a random number generator defined by the `Random` class. To use this class, it must be instantiated. The call to the constructor determines the initial seed of the random numbers. If no parameter is provided, the seed is the time of day clock of the system. If a parameter is given, it is used as the seed. For example, in the following, the clock is used as the seed:

```
Random myRand = new Random();
```

To create a random number, the `Next` method of `Random` is called. The `Next` method can take zero, one, or two parameters. If no parameter is given, it generates an integer in the range of zero to `Int32.MaxValue`, which is the largest positive integer that can be represented in 32 bits, which is 2,147,483,647. If

one parameter is passed to `Next`, say m, it generates numbers in the range of $0..m-1$. If two parameters are passed, say m and n, `Next` generates a number in the range of $m..n-1$..

```
<!-- ex1.aspx
     A simple example of an ASP.NET document
     It uses a function to fill an array with pseudorandom numbers,
     which are then displayed
     -->
<%@ Page language="c#" %>
<html>
  <head> <title> Ex1 </title>
    <script runat = "server">

// Build a pseudorandom number method
      Random randomGen = new Random();
      string msg;
      private int [] myArray = new int[10];

// A method to fill the array with pseudorandom numbers
      public void fillArray() {
        for (int index = 0; index < 10; index++)

// Generate a pseudorandom number in the range of 0 to 99
          myArray[index] = randomGen.Next(0, 100);
      }
    </script>
  </head>
  <body>

<!-- Code to call the fillArray method and display the array -->
    <% fillArray();
       Response.Write(
         "<br /> <b>The array's contents are: </b><br /><br />");
       for (int index = 0; index < 10; index++) {
         msg = string.Format("The element at {0} is: {1} <br />",
                             index, myArray[index]);
         Response.Write(msg);
       }
    %>
  </body>
</html>
```

12.3.3 Code-Behind Files

As stated in Section 12.3.1, it is better to keep programming code separate from documents, which also separates program logic from presentation. This is done in ASP.NET by storing programming code in code-behind files.

To illustrate the difference between declaration blocks and code-behind files, `ex1.aspx` is rewritten as `ex2.aspx` and the code-behind file `ex2.aspx.cs`, which are shown below. Notice that the `@Page` directive in this ASP.NET document includes two new attributes. The `Inherits` attribute specifies that this document inherits from its code-behind file's class. The name used for this attribute is the same as the base name of this file. Also included is the `Src` attribute, which gives the full name of the code-behind file. When the `Src` attribute is included, the code-behind file is implicitly compiled the first time its associated ASP.NET document is requested. If the code-behind file is updated, the next request for the associated ASP.NET file implicitly causes its recompilation.

If the `Src` attribute is absent, the code-behind file must be explicitly compiled and placed in the `bin` subdirectory of the directory in which the ASP.NET document is stored, before the associated ASP.NET document is requested. This approach has the advantage of allowing the detection and repair of syntax errors in the code-behind file before deployment.

```
<!-- ex2.aspx
     A simple example of an ASP.NET document with a code-behind file
     It has the same functionality as ex1.aspx
     -->
<%@ Page language="C#" Inherits = "ex2" Src = "ex2.aspx.cs" %>
<html>
  <head> <title> Ex2 </title>
  </head>
  <body>

<!-- Code to call the fillArray method and display the array
    <% string msg;
       fillArray();
       Response.Write(
         "<br /> <b>The array's contents are: </b><br /><br />");
       for (int index = 0; index < 10; index++) {
         msg = string.Format("The element at {0} is: {1} <br />",
                             index, myArray[index]);
         Response.Write(msg);
       }
    %>
  </body>
</html>
```

```
// ex2.aspx.cs
//   The code behind file for ex2.aspx
//   Includes a function to build an array of ten pseudorandom
//   numbers
using System;
using System.Web;
using System.Web.UI;

public class ex2 : Page {

// Build a pseudorandom number method
  Random randomGen = new Random();
  protected int [] myArray = new int[10];

// A method to fill the array with pseudorandom numbers
  public void fillArray() {
    for (int index = 0; index < 10; index++)

// Generate a pseudorandom number in the range of 0 to 100
      myArray[index] = randomGen.Next(0, 100);
  }
}
```

12.4 ASP.NET Controls

ASP.NET controls are related to XHTML elements, but they have associated program code that is executed on the server. Therefore, they are called *server controls*. There are two categories of ASP.NET server controls, HTML controls and Web controls.

12.4.1 HTML Controls

The ASP.NET namespace `System.Web.UI.HTMLControls`, which is implicitly available to all ASP.NET documents, includes the HTML controls. HTML controls are closely related to XHTML form elements. However, HTML controls were designed to allow interactions with server-side programming code. These interactions include the possibility of server-side programming code that can dynamically change the appearance and functionality of HTML controls. This is possible because the attributes that control these are exposed by the objects that represent the controls in the document class. Furthermore, many HTML controls can raise one of two events when they are manipulated by the client. These two events are `ServerClick`, which corresponds to the JavaScript click event, and `ServerChange`, which corresponds to the JavaScript

change event. The most commonly used HTML control types, their corresponding XHTML elements, and the events they raise are shown in Table 12.2.

Table 12.2 HTML control types, corresponding XHTML elements, and their events

Control Type	XHTML Element	Event
`HtmlHidden`	`<input type = "hidden">`	`ServerChange`
`HmtlInputButton`	`<input type = "button">`	`ServerClick`
	`<input type = "submit">`	`ServerClick`
	`<input type = "reset">`	`ServerClick`
`HtmlInputCheckBox`	`<input type = "checkbox">`	`ServerChange`
`HtmlInputRadioButton`	`<input type = "radio">`	`ServerChange`
`HtmlInputText`	`<input type = "text">`	`ServerChange`
	`<input type = "password">`	`ServerChange`
`HtmlAnchor`	`<a>`	`ServerClick`
`HtmlButton`	`<button>`	`ServerClick`
`HtmlSelect`	`<select>`	`ServerChange`
`HtmlTextArea`	`<textarea>`	`ServerChange`
`HtmlForm`	`<form>`	None
`HtmlTable`	`<table>`	None
`HtmlTableCell`	`<td> and <th>`	None
`HtmlTableRow`	`<tr>`	None
`HtmlGenericControl`	Nonform elements	None

Notice that some of the HTML control types are not form elements, for example, `HTMLTable`. Users cannot interact with them. They have types to provide program access to their attributes to allow them to be changed dynamically.

Server-side controls that appear in ASP.NET documents compile into fields or members of the document class. The `id` of the control becomes the name of the field in the document class. Therefore, all server-side controls must have `id` attributes. Also, a control is specified to be a server-side control by including the `runat` attribute set to `"server"`. Without this, the control is just a static XHTML widget. For example, suppose the document includes the following:

```
<form runat = "server">
```

```
  <input type = "text"  id = "address"
         runat = "server" />
  ...
</form>
```

For this form, the document class would have the following field:

```
protected HtmlInputText address;
```

Notice that the `<form>` tag in the preceding example does not include an `action` attribute. This is the case in all ASP.NET documents. The reason is that all programming actions that result from user interactions with controls are defined in the ASP.NET document itself or in its code-behind file.

The HTML control class is derived from the generic `HtmlControl` class, from which the methods and properties that are used for all HTML controls are inherited. For example, the `Attributes` collection has a name/value pair for each attribute of the element. Methods and properties that are specific for an element are defined in the element's class. For example, the `HtmlAnchor` class includes the `Href` property.

The difference between HTML controls and ordinary static XHTML widgets is that the HTML controls are converted to objects in the document class. This means the controls and their attributes, which includes their styles, are available to and can be modified by the code of the document. As previously stated, an XHTML widget is converted to an HTML control simply by adding the `runat` attribute set to `"server"`. For example, consider the following XHTML text box:

```
<input type = "text"  id = "address" />
```

The following is the same text box, but as an HTML control:

```
<input type = "text"  id = "address"  runat = "server" />
```

12.4.2 Life Cycle of a Simple ASP.NET Document

An ASP.NET document that includes a form serves both to describe the initial content of an XHTML document for browser display and to provide the event handling to process user interactions with the form in the document. So, for all ASP.NET documents that include forms, there are two kinds of requests. First, there is an initial request, which results in the requested document and its form being displayed for the client. Second, there is a request made after the form has been changed by the client. This kind of request is called a *postback* because the form values are posted back to the document on the server. C# code in a document can determine whether it is a postback request by testing the `IsPostBack` property, which is true if it is a postback request.

C# code in a document can access the values of controls through the `Value` property of the associated object. The object associated with a control has the same name as the control's id attribute. So, if a form has a text box with the id `phone`, its value can be accessed as `phone.Value`.

To clarify the sequence of events that takes place for an ASP.NET document that includes a form, consider the following simple document example:

```
<!-- ex3.aspx
     A simple example of an ASP.NET document with HTML controls.
     It uses text boxes to get the name and age of the client,
     which are then displayed.
     -->
<%@ Page language="c#" %>

<html>
  <head> <title> Ex3 </title>
  </head>
  <body>
    <form runat = "server">
      <p>
        Your name:
        <input type = "text" id = "name" runat = "server" />
        <br />
        Your age:
        <input type = "text" id = "age" runat = "server" />
        <br />
        <input type = "submit" value = "Submit" />
      <br />
      <% if (IsPostBack) { %>
        Hello <%= name.Value %> <br />
        You are <%= age.Value %> years old <br />
        <% } %>
      </p>
    </form>
  </body>
</html>
```

Notice that both the form and the controls in the form must include the `runat` attribute set to "`server`".

ASP.NET implicitly stores the control state of a document class instance before the server returns the output of the instance to the client. This information is stored in a hidden control, which is a property of the `Page` class, named `ViewState`. `ViewState` is a reference to a `StateBag` object, which is a data structure similar to a hash. `StateBag` objects are only valid while the page is active. If the browser is pointed at a different document, the `StateBag` object is discarded. When the document is posted back to the server, the `ViewState` data is used to initialize the new instance implicitly. Of course, `ViewState` will not have form data on the first postback. After initialization using `ViewState`,

the client input from the form is used for a second initialization of the instance. Therefore, any widget whose value is not input by the client retains its previous value. `ViewState` provides implicit form state preservation between requests. So, it does what the HTTP protocol cannot do, save state across the round trips to the server.

Of course, saving state with `ViewState` is not free. For a large form with many controls, the resulting `ViewState` will require more time for browser/server communications, as well as significant storage space on the client machine.

The following is a list of the things that happen if the `ex3.aspx` document is requested, is delivered to the browser, has its text boxes filled in by the user, is posted back to the server, and finally is returned to the browser. Note that many events are raised during this processing, although none is described in the following:

1. The client requests `ex3.aspx`, the original ASP.NET document.
2. A document class is created by compiling the requested document. Then the constructor of that class is invoked.
3. The control state of the instance is initialized with the `ViewState` data. (On the initial request, there is no `ViewState` data.)
4. The form data of the request is used to set the control state of the document class instance. (On the initial request, there is no form data.)
5. The current state of the instance is recorded in the `ViewState` hidden field.
6. The instance is executed and the results returned to the client.
7. The class and its instance are deleted on the server.
8. The client interacts with the form of the document.
9. The client causes a postback to the server.
10. A document class is created by compiling the requested document. Then the constructor of that class is invoked.
11. The control state of the instance is initialized with the `ViewState` data.
12. The form data of the request is used to set the control state of the document class instance.
13. The current state of the class is recorded in `ViewState`.
14. The instance is executed and the results are returned to the client; the class and its instance are deleted on the server.

`ViewState` is user-accessible, so it can be used to store state information other than form data. All controls inherit `ViewState` from the `Controls` class. Any textual data can be placed in `ViewState` with a simple assignment statement, for example:

```
ViewState["myName"] = "Freddie";
```

Accessing the values in `ViewState` is slightly complicated by the necessity of casting the value to the proper type. For example, to fetch the `myName` value above, the following could be used:

```
name = (string)ViewState["myName"];
```

To use `ViewState` for non-form data, that data must be assigned to `ViewState` before Step 13, because it is at that point that `ViewState` is finalized. In Section 12.4.3, the `PreRender` event is introduced, which is the perfect time to record non-form data in `ViewState`.

The document created by the document class that was compiled from the `ex3.aspx` document, after it has had its form filled by the client, is as follows:

```
<!-- ex3.aspx
     A simple example of an ASP.NET document with HTML controls.
     It uses text boxes to get the name and age of the client,
     which are then displayed.
     -->

<html>
  <head> <title> Ex3 </title>
  </head>
  <body>
    <form name="_ct10" method="post" action="ex3.aspx id="_ct10">
 <input type="hidden" name="_VIEWSTATE"
 value="dDwxMTYxOTE0NDI7Oz4rK1loC8s5yMYUcQVQuGFG7BGQzg==" />

      <p>
        Your name:
        <input type = "text" id = "name" runat = "server" />
        <br />
        Your age:
        <input type = "text" id = "age" runat = "server" />
        <br />
        <input type = "submit" value = "Submit" />
      <br />

        Hello mike <br />
        You are 47 years old <br />

      </p>
    </form>
  </body>
</html>
```

This document differs from the original version of `ex3.aspx` in three areas. First, it includes the `ViewState` hidden control, which has a coded version of the form data. The code used is base64. Second, the form has an internal name and ID (`_ct10`). Third, the render block to produce the return XHTML has been replaced by its output.

A postback can be initiated by a user in more than one way. Of course, a postback occurs if the user clicks the *Submit* button of a form. It also happens when any button is clicked. You have the option of having a postback happen when a checkbox is clicked or a select item is selected. This is controlled by the `AutoPostBack` property of the control. If `AutoPostBack` is set to `"true"`, then a change in the control's value causes a postback.

12.4.3 Page-Level Events

There are two levels of events that can be raised while an ASP.NET document is being processed. First, there are the control events, `ServerClick` and `ServerChange`, raised by the HTML controls and discussed in Section 12.4.4. Second, there are the four page-level events. These events actually are created by the `Page` class. These are `Init`, which is raised immediately after a document class is instantiated; `Load`, which is raised just after the instance has its state set from form data and `ViewState`; `PreRender`, which is raised just before the instance is executed to construct the client response document; and `Unload`, which is raised just before the instance is discarded.

There are two ways to design and register handlers for the page-level events. The first is to write the handlers using predefined names that are implicitly registered when the document class is created. This implicit handler registration is called *auto event wireup*. It is controlled by the `@Page` directive attribute, `AutoEventWireup`, which has the default value of `true`. If set to `false`, the implicit registration is not done, and registration must be done manually. The names of the handlers that are implicitly registered are `Page_Load`, `Page_Unload`, `Page_PreRender`, and `Page_Init`. All return `void` and take a parameter of type `System.EventArgs`. The protocols of these are as follows:

```
public void Page_Unload(System.EventArgs e) { ... }
public void Page_Load(System.EventArgs e) { ... }
public void Page_PreRender(System.EventArgs e) { ... }
public void Page_Init(System.EventArgs e) { ... }
```

`Page_Init` is used in an example in Section 12.4.4.

The second way to design and register event handlers for page-level events is to override the virtual handler methods defined in the `Page` class. Such handlers must be manually registered in the document. This approach is not further discussed here.

12.4.4 Control Events

There are two ways to create and register event handlers for HTML controls. The first of these is similar to the way JavaScript client-side event handlers are registered through XHTML attributes. The attributes are named `OnServerClick` and `OnServerChange`, which correspond to the two control events for HTML controls. The handlers for HTML controls all use the following protocol: They return `void` and take two parameters, the first of type `object` and the second of type `System.EventArgs`. This is the protocol for the `EventHandler` delegate, which provides the standard event handling approach for CLR. For example, consider the following event handler for a text box control, along with the control:

```
protected void TextboxHandler(object src,
                              System.EventArgs e) {
  ...
}
...
<input type = "text"  id = "Name"
       OnServerChange = "TextBoxHandler"
       runat = "server" />
```

The second way to write and register event handlers is to use the standard CLR approach, which uses delegates for handler registration. The events are of the generic `EventHandler` delegate type. There are three steps to creating and registering event handlers: First, the event handler is written as a function with the return type and parameter types of `EventHandler`. Second, a new instance of the delegate type is created using `new`, passing the name of the event handling function to the constructor. Third, the delegate instance must be subscribed to the event property of the control by adding it to any delegates already subscribed there. The second and third parts of this process are usually combined into one statement and placed in a `Page_Init` handler so that they are accomplished before the document class instance is executed. In the following example, the same text box and handler as previously shown are repeated, this time using delegates.

```
// The event handler
protected void TextBoxHandler(object src,
                              System.EventArgs e) {
  ...
}

// Use a Page_Init handler to create and subscribe the
// handler
protected void Page_Init(object src, EventArgs e) {
  Name.ServerChange +=
                 new EventHandler(TextboxHandler);
}
```

```
...
<!-- The text box -->
<input type = "text"  id = "Name"  runat = "server" />
```

We can now revise the life cycle of an ASP.NET document request to include event creation for both page-level and control events. This time we list only a single request cycle (rather than including a postback cycle).

1. The client requests `ex3.aspx`, the original ASP.NET document.
2. A document class is created by compiling the requested document. Then the constructor of that class is invoked.
3. The `Page` event `Init` is raised.
4. The control state of the instance is initialized with the `ViewState` data. (On the initial request, there is no `ViewState` data.)
5. The form data of the request is used to set the control state of the document class instance. (On the initial request, there is no form data.)
6. The `Page` event `Load` is raised.
7. Server-side control events are raised.
8. The `Page` event `PreRender` is raised.
9. The current control state of the instance is recorded in the `ViewState` hidden field.
10. The instance is executed and the results returned to the client.
11. The `Page` event `Unload` is raised.
12. The class and its instance are deleted on the server.

12.4.5 Web Controls

Web controls are based on the controls of VB. The `System.Web.UI.WebControls` namespace includes the Web controls. This is a larger and richer collection of controls than the HTML controls. In addition to controls that correspond to the ordinary XHTML widgets, there are many more. For example, there are controls for checkbox lists, radio button lists, drop-down lists, and list boxes. In addition, there are special controls for form data validation and data binding.

The complete one-to-one correspondence between HTML controls and the XHTML widgets is absent with the Web controls. The most commonly used Web control types are shown in Table 12.3.

Some additional Web control types are described in the following paragraphs. All of the Web controls are in the `asp` namespace, so the tag names are all qualified with `asp:`. For example, a text box control is specified with the following:

```
<asp:textbox id = "phone"  runat = "server" />
```

In addition to having a more consistent programming interface than the HTML controls, the Web controls collection provides controls that do not cor-

Table 12.3 Commonly used Web controls and related XHTML elements

Web Control Type	XHTML Element
`AdRotator`	None

`Button`	`<input type = "button" />`
	`<input type = "submit" />`
	`<input type = "reset" />`

`Calender`	None
`Checkbox`	`<input type = "checkbox" />`
`CheckBoxList`	None
`DropDownList`	`<select>`
`Label`	None
`RadioButton`	`<input type = "radio" />`
`RadioButtonList`	None
`Table`	`<table>`
`TableCell`	`<th>, <td>`
`TableRow`	`<tr>`
`TextBox`	`<input type = "text" />`

respond to XHTML widgets and are rendered as combinations of widgets. Among these are `Xml`, `Panel`, `AdRotator`, and the list controls. The `Xml` control provides the ability to include XSL transformations on XML input as part of the output XHTML document. The `Panel` control provides a container for other controls, for those situations where you want to control the position or visibility of the contained controls as a unit. The `AdRotator` provides a way to implicitly produce different content on different requests.

The `ListControl` class has four subclass controls. Two of these are familiar, `DropDownList` and `ListBox`, and both are converted to XHTML select elements. The `ListBox` control can display one or more of its items. The number of display items defaults to four but can be set to any number. A vertical scrollbar is implicitly included if the control has more items than the number it can display. More than one item in a `ListBox` can be selected. The `DropDownList` control remains hidden until the user clicks its button. The browser chooses the number of items displayed when the drop-down button is clicked. `DropDownList` controls do not allow multiselection mode.

The two other `ListControl` subclass controls are `CheckBoxList` and `RadioButtonList`. Both of these are normally translated to table XHTML elements. In both cases, the purpose is to allow programming code access to the

list items in the lists. This supports the possibility of adding and/or deleting list items dynamically as the result of user interaction. It also makes it possible for list items to be fetched from a database or other external source.

12.4.6 Creating Control Elements with Code

Server-side controls can be specified for an ASP.NET document in two different ways: with markup or with programing code. For example, a button can be created with the following markup:

```
<asp:button id = "helpButton"  Text = "help"
            OnClick = "OnClickHandler"
            runat = "server" />
```

The same button could be created with C# code, as shown in the following:

```
protected Button helpButton = new Button();
helpButton.Text = "help";
helpButton.id = "helpButton";
helpButton.OnClick = "OnClickHandler";
helpButton.runat = "server";
```

There are two problems with creating controls with program code: First, it requires more typing. Second, the placement of the control on the document display is problematic. It has to be added to something already in the document. To control the placement, a placeholder element can be defined in the markup. Then the control can be added using the id attribute of the placeholder. This gives the exact position within the document for the control. For example, the placeholder could be specified with the following:

```
<asp:placeholder id = "buttonPlace"  runat = "server" />
```

The following statement places the button at the position where the placeholder element appeared:

```
buttonPlace.Controls.Add(helpButton);
```

More than one control can be put in a placeholder. They are maintained in a property of the placeholder element, `Controls`. So, the `Controls` property is a collection of controls elements. The order in which controls are added to the placeholder's `Controls` property determines the order in which the controls will appear in the display.

Although it is easier to create elements with markup, modifying elements is a good use of program code. For example, the list items of a select element could be added with program code, after the select element had been specified in markup. This would be especially useful if the list items came from some other data source. Program code is also useful for modifying the attributes of a markup-created element.

12.4.7 Response Output for Controls

The first two sample ASP.NET documents of this chapter use the `Response.Write` method to place text in the response buffer. This is not a viable approach when there are controls in the document because the output from `Response.Write` goes to the beginning of the buffer rather than the position among the controls of the call to `Response.Write` (assuming the code is embedded in an ASP.NET document). As a more effective alternative, the text can be placed in a label control, which produces the text at the position of the label control in the response buffer. The text is assigned to the `Text` property of the label control. For example, suppose the document includes the following (at the position where the output text should be):

```
<asp:label id = "output"  runat = "server" />
```

The following places the given text at that position of the label in the response buffer:

```
<% string msg = string.Format(
                "The result is {0} <br />", result);
output.Text = msg; %>
```

In this example, the `string.Format` method is used to create a formatted string that consists of literal text and the value of the variable, `result`, which has been converted to text by `Format`. Of course, the program code could also appear in a code-behind file.

12.4.8 An Example

The following example creates a text box, a drop-down list, and a button in an ASP.NET document. It uses code in a code-behind file to fill in the list items of the drop-down list. The document also includes a label control to provide a place for the return message from the code-behind file. The code-behind file also includes a handler for the button, which confirms to the user the selected item.

The list items are added to the select element with the `Add` method of the `Items` property of the select. Each new item is created with a call to the list item constructor, `ListItem`, passing the value of the new item. For example, to add a list item with the value `"red"` to the select control with the id `mySelect`, the following could be used:

```
mySelect.Items.Add(new ListItem("red"));
```

The button handler in the example will return a message to the client, giving his or her name and the chosen select item, which in this case is a color. The client name is retrieved from the `name` text box of the document, using the `Text` property of the text box. The chosen color is retrieved from the form with the `SelectedItem` property of the drop-down list.

The ASP.NET document and its code-behind file follow:

```
<!-- ex4.aspx
     An example of an ASP.NET document that creates a text box,
      a drop-down list, a submit button, and a label.
     A code-behind file is used to populate the drop-down list and
      handle the button clicks. The label is used for the return
      message
      -->
<%@ Page language="c#" Inherits = "ex4" Src = "ex4.aspx.cs" %>
<html>
  <head> <title> Ex4 </title>
  </head>
  <body>
    <form runat = "server">
      Name: <asp:textbox runat = "server" id = "name" />
      <br /><br />
      Favorite Color:<asp:DropDownList runat = "server"
                                       id = "color" />
      <br /><br />
      <asp:button  runat = "server" id = "submit"
                   text = "Submit" OnClick = "OnClickHandler" />
      <br /><br />
      <asp:label id = "message" runat = "server" />
    </form>
  </body>
</html>
```

```
// ex4.aspx.cs
//   The code behind file for ex4.aspx.
//   In an OnLoad handler, it populates the drop-down
//   list created in the associated ASP.NET document.
//   It also includes a handler for the button, which
//   produces a message to the client, including the
//   client's name and the chosen item from the drop
//   down list
using System;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public class ex4 : System.Web.UI.Page {
  protected DropDownList color;
```

```
  protected TextBox name;
  protected Button submit;
  protected Label message;

// OnLoad handler to populate the dropdownlist
  override protected void OnLoad(EventArgs e) {
    if (!IsPostBack) {
      color.Items.Add(new ListItem("blue"));
      color.Items.Add(new ListItem("red"));
      color.Items.Add(new ListItem("green"));
      color.Items.Add(new ListItem("yellow"));
    }
  }

// Handler for the button
  protected void OnClickHandler(object src, EventArgs e) {
    string newMsg = string.Format(
        "Hi {0}, your favorite color is {1}",
        name.Text, color.SelectedItem);
    message.Text = newMsg;
  }
}
```

12.4.9 Validation Controls

In Chapter 5, "JavaScript and HTML Documents," client-side form data validation with JavaScript was discussed. Although there are strong reasons for doing form data validation on the client, there are also reasons to do it again on the server. First among these is that client-side validation can be subverted by a devious client. In some cases, form data goes directly into a database, which could be corrupted by bad data. So, it is often necessary to do form data validation on both the client and the server side. In the following paragraphs, we introduce the ASP.NET Web controls designed to make server-side form data validation relatively easy.

There are six validation controls defined in the Web controls collection. The four most commonly used of these controls, along with their properties and values, are shown in Table 12.4.

The two validation controls that are not shown in Table 12.4 are one for custom validation, which is done in functions, and one producing a summary of all of the error messages produced during server-side validation.

Validation controls are placed immediately after the controls whose values they are to validate. This placement is necessary so the error messages produced by the validation controls appear next to the controls being validated. The actual error message is specified in the `ErrorMessage` attribute of the validation control. The validation control is connected to the control it is to validate

Table 12.4 Validation controls and their properties

Control	Properties	Values
`RequiredFieldValidator`	None	None
`CompareValidator`	`Operator`	`Equal`, `NotEqual`, `GreaterThan`, `GreaterThanEqual`, `LessThan`, `LessThanEqual`, `DataTypeCheck`
	`Type`	`String`, `Currency`, `Date`, `Double`, `Integer`
	`ValueToCompare`	Constant
	`ControlToCompare`	Another control
`RangeValidator`	`MaximumValue`	Constant
	`MinimumValue`	Constant
	`Type`	`String`, `Currency`, `Date`, `Double`, `Integer`
`RegularExpressionValidator`	`ValidationExpression`	Regular expression

with the `ControlToValidate` property, which is set to the id of the control. The `Display` property is used to specify how the error message will be displayed. The value "`Static`" means that space is reserved on the displayed document for the message. The value "`Dynamic`" means space for the message is not reserved. The value "`None`" means no error message will be displayed, although the error is still recorded in a log. Validation controls must also include the `runat` attribute set, of course, to "`server`".

The following example, `ex5.aspx`, illustrates some of the validation controls.

```
<!-- ex5.aspx
     An example of an ASP.NET document to illustrate server-side
     validation Web controls.
     It uses Web control text boxes to get the name, phone number,
     and age of the client. These three are validated on the
     server
     1. The name must be present
     2. The phone number must be in the form ddd-ddd-dddd
     3. The range of the age must be 10 to 110
     -->
<%@ Page language="c#" %>
```

```
<html>
  <head> <title> Ex5 </title>
  </head>
  <body>
    <form runat = "server">
      <p>
        Your name:
        <asp:textbox  id = "name" runat = "server" />
        <asp:RequiredFieldValidator
          ControlToValidate = "name"
          Display = "Static"
          runat = "server"
          ErrorMessage = "Please enter your name">
        </asp:RequiredFieldValidator>
        <br />

        Your phone number:
        <asp:textbox  id = "phone" runat = "server" />
        <asp:RegularExpressionValidator
          ControlToValidate = "phone"
          Display = "Static"
          runat = "server"
          ErrorMessage = "Phone number form must be ddd-ddd-dddd"
          ValidationExpression = "\d{3}-\d{3}-\d{4}">
        </asp:RegularExpressionValidator>
        <br />

        Your age:
        <asp:textbox  id = "age" runat = "server" />
        <asp:RangeValidator
          ControlToValidate = "age"
          Display = "Static"
          runat = "server"
          MaximumValue = "110"
          MinimumValue = "10"
          Type = "Integer"
          ErrorMessage = "Age must be in the range of 10 to 110">
        </asp:RangeValidator>
        <br />

        <input type = "submit" value = "Submit" />
      </p>
    </form>
  </body>
</html>
```

The name text box is validated to ensure that a name is given. The phone number text box is validated to ensure that it matches the given regular expression. (Regular expressions are described in Chapter 4, "The Basics of JavaScript.") The age text box is validated to ensure that the given age is at least 10 but not greater than 110.

Figure 12.2 shows the display of the `ex4.aspx` document after some of the fields have been filled incorrectly, which results in the appearance of error messages to the right of the text boxes.

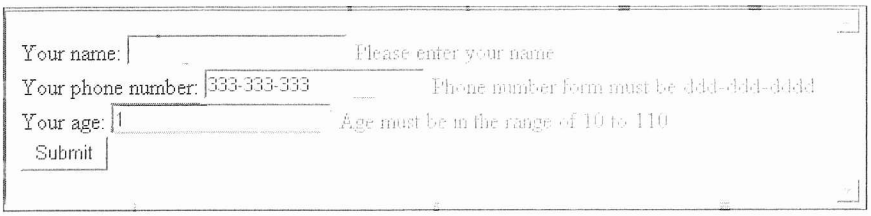


Figure 12.2 Display of `ex4.aspx` after some text boxes have been filled

12.5 Web Services

Web services, which were introduced in Chapter 7, "Introduction to XML," use technologies that have only recently been developed. In brief, a Web service is a collection of one or more related methods that can be called by remote systems using standard protocols on the Web. Microsoft is one of the companies that provides Web services. The most widely used Microsoft Web service is Passport, which identifies and authenticates users, among other things.

The .NET Framework provides several kinds of support for the construction and advertisement of Web services. The most powerful support tool is Visual Studio .NET, which is not discussed in this book. However, even without Visual Studio .NET, it is relatively easy to create and advertise Web services with the .NET Framework.

12.5.1 Constructing Web Services

In .NET, a Web service is simply a special kind of class, which can be written in any .NET programming language. The Web service class can be stored in a file with the extension `.asmx` (just like any ASP.NET Web application). In most cases, however, the `.asmx` file stores only a directive—the code is in a code-behind file with a further extension in its name. If the code is written in C#, the extension on the code-behind file would be `.asmx.cs`. The structure of the directive in the `.asmx` file is illustrated in the following example:

```
<%@ WebService Language = "C#"
               Codebehind = "Service1.asmx.cs"
               Class = "MyWebService1.Service1" %>
```

This example directive, which is in the file named `Service1.asmx`, indicates the Web service is a class written in C#, whose code-behind file is named `Service1.asmx.cs`. The class itself is named `Service1` and is in the `MyWebService1` namespace.

Following is a simple Web service class, `Service1.asmx.cs`, which defines a service method `Sum3` that takes three integers as parameters and returns the sum of the three given numbers.

```
using System;
using System.Web.Services;

namespace MyWebService1 {
[WebService(Namespace =
       "http://www.sebesta.com/webservices/")]

  public class Service1 :
         System.Web.Services.WebService {
    [WebMethod]
    public int Sum3(int first, int second, int third) {
      int sum;
      sum = first + second + third;
      return sum;
    }
  }
}
```

This code-behind file imports `System.Web.Services` and inherits from the `WebService` class, which is defined in `System.Web.Services`. The `WebService` class provides various kinds of support for Web services, including the `WebMethodAttribute` class and the `Context` object, which contains information about the HTTP request that invokes the Web service.

The line that begins `"[WebService"` is used to define a namespace for Web services on this server. If it is not included, a default generic namespace is used, which could cause conflicts with other Web services. The one method of `Service1` is marked `[WebMethod]` to specify that it is to be made available as a Web service. `WebMethod` is an attribute of the `WebMethodAttribute` class.[4] A Web service class can include methods that are not Web services—they are those that are not marked with `[WebMethod]`.

An `.asmx` file that defines a Web service can be viewed with an IE browser, which illustrates some of what the .NET system creates when a Web service class file is built. The display of the example Web service is shown in Figure 12.3.

4. An attribute is used to indicate something special about a declaration.

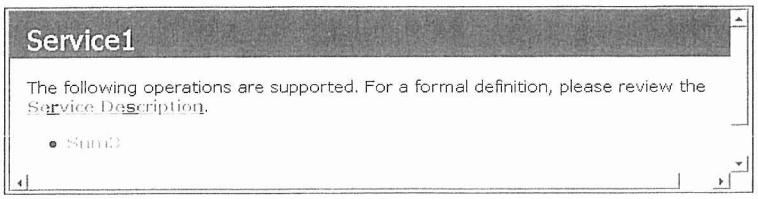


Figure 12.3 An IE browser display of the `Service1`

Figure 12.3 shows that the display includes a link (`Service Description`) to a Web Service Description Language (WSDL) description of the Web service, which has the required formats for requests and responses to the methods of the Web service. Recall that WSDL is an XML-based markup language for defining the protocols of Web services. The formats given in the service description are provided for the Simple Object Access Protocol (SOAP), HTTP GET, and HTTP POST protocols. SOAP is also an XML-based markup language. The first part of the service description for our example is shown in Figure 12.4.

```
<?xml version="1.0" encoding="utf-8" ?>
- <definitions
    xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:s="http://www.w3.org/2001/XMLSchema"
    xmlns:s0="http://www.sebesta.com/webservices/"
    xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
    xmlns:tm="http://microsoft.com/wsdl/mime/textMatching/"
    xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
    targetNamespace="http://www.sebesta.com/webservices/"
    xmlns="http://schemas.xmlsoap.org/wsdl/">
  - <types>
    - <s:schema elementFormDefault="qualified"
        targetNamespace="http://www.sebesta.com/webservices/">
      - <s:element name="Sum3">
        - <s:complexType>
          - <s:sequence>
              <s:element minOccurs="1" maxOccurs="1"
                name="first" type="s:int" />
              <s:element minOccurs="1" maxOccurs="1"
                name="second" type="s:int" />
              <s:element minOccurs="1" maxOccurs="1"
                name="third" type="s:int" />
            </s:sequence>
          </s:complexType>
        </s:element>
      - <s:element name="Sum3Response">
        - <s:complexType>
          - <s:sequence>
              <s:element minOccurs="1" maxOccurs="1"
                name="Sum3Result" type="s:int" />
            </s:sequence>
          </s:complexType>
        </s:element>
        <s:element name="int" type="s:int" />
      </s:schema>
    </types>
```

Figure 12.4 The first part of a service description

Also displayed in Figure 12.3 are the names of the Web service methods. Our example has just one method, `Sum3`. Each method name is a link to a test document for the method. Parameters can be given to the tester, which runs the method and displays the results when the `Invoke` button is clicked. Figure 12.5 shows the tester for the `Sum3` method, including test values.

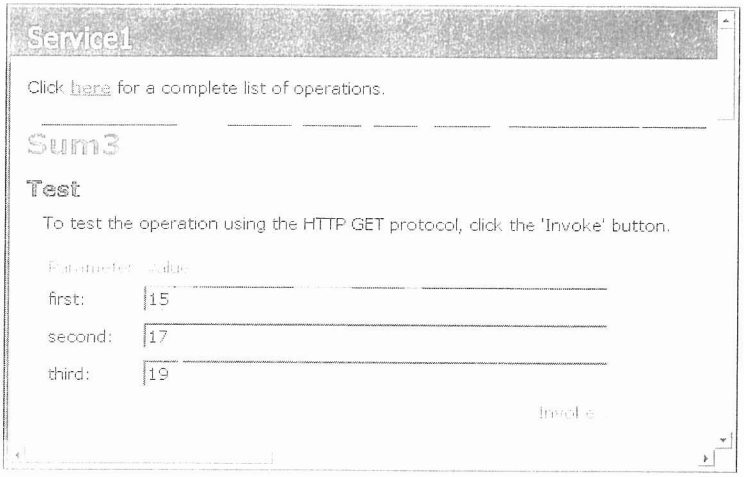


Figure 12.5 The test display for the `Sum3` service

Clicking the `Invoke` button of the `Sum3` tester produces the screen display shown in Figure 12.6.

Figure 12.6 The result of invoking the `Sum3` tester

12.6.2 Advertising Web Services

If a client has the URL of a Web service, he or she can easily use it. However, the more general situation is that a potential client does not know the URL of a possibly useful Web service. There are two approaches used with .NET to make Web services available to clients: with a Web services discovery document and with a Web services directory written with the Universal Description, Discovery, and Integration (UDDI) language (UDDI was introduced in Section 7.11). In both cases, a directory of all Web services provided by a Web site can be made available to potential clients through a single URL on the site.

UDDI is part of Windows in the .NET Server release. It can be used to set up a UDDI server for inside an enterprise, as well as to register electronic ser-

vices to make them available to the outside world. These activities are supported by the .NET UDDI SDK, which includes documentation, the `Microsoft.Uddi` assembly for the .NET Framework, and several example applications.

Summary

.NET is a collection of technologies that supports the development and deployment of distributed component-based software systems written in variety of languages. The .NET Framework is a generic support structure for the .NET family of languages. The CLR is a runtime system, including JIT compilers that support the execution of .NET software. There are now 20 .NET languages, with more on the way. The CTS defines a set of types that must be supported by .NET languages. The CLS defines a minimal set of language features that must be supported by .NET languages. Software in any .NET language can interact in a variety of ways with software written in any of the other .NET languages.

The primary .NET language, C#, was designed specifically for the .NET system. C# is based on Java but includes some features of other languages, notably C++, VB, and Delphi, as well as some new language features. Among its features are a type-safe enumeration type, an improved `switch` construct, a `foreach` statement, some new controls on method inheritance, a value type struct, and properties.

ASP.NET is an approach to server-side support of dynamic documents. It is similar to JSP but is language neutral. Programming code can reside in an ASP.NET document or in a separate file called a code-behind file. In either case, the code is compiled before it is executed. Every ASP.NET document is compiled into a class before it is used, regardless of whether it contains programming code. All such classes are subclasses of the predefined class, `Page`, unless they have code-behind files, in which case the code-behind file inherits from `Page` and the class for the ASP.NET document inherits from the code-behind file.

ASP.NET documents consist of XHTML, programming code (either in script elements or render blocks), directives, server-side comments, and server-side controls. Server-side controls include the `runat` attribute set to `"server"`. The only required directive is `@Page`, which must include the `Language` attribute, which specifies the language used for the programming code, either embedded or in a code-behind file. A code-behind file can be either precompiled or dynamically compiled only after it has been changed and the associated ASP.NET document has been requested. Just as JavaBeans is the best way to use Java in a dynamic document, code-behind files are the best way to use a .NET language to support dynamic documents.

There are two categories of ASP.NET controls, HTML controls and Web controls. HTML controls closely parallel the widgets of XHTML. These controls result in objects in the compiled `Page`-derived class, whereas the static XHTML code of a document is simply emitted by the execution of the `Page`-

derived class. An HTML control can create one of the two events, `ServerClick` or `ServerChange`, which can be handled by server-side code. The `id` attribute value of an HTML control becomes the associated variable's name in the compiled version of the document.

The state of an ASP.NET document is implicitly maintained between requests with the `ViewState` hidden field.

There are four page-level events defined in the `Page` class: `Init`, `Load`, `Unload`, and `PreRender`. These events can be handled in server-side code. The handlers can be implicitly registered by naming them with predefined names and using the proper protocol. Alternatively, they can be subscribed to the event handler delegate, `EventHandler`.

Control event handlers can be registered in two ways, either by referencing them on an attribute on the control or by using the `EventHandler` approach that can be used for page-level event handlers.

The Web controls collection is richer than the HTML controls collection, and they make programming interactions easier.

The .NET Framework provides significant assistance in building Web services.

Review Questions

12.1 What is a component?

12.2 What is the difference between a JavaBean and a .NET component?

12.3 When does a JIT compiler perform its translation of a method?

12.4 What is the primary benefit of the multilanguage aspect of .NET?

12.5 What part of the .NET system controls the execution of programs?

12.6 Explain how a JIT compiler works.

12.7 Describe briefly the two parts of the CLI.

12.8 On what languages is C# based?

12.9 Explain two reasons why C# enumeration types are safer than those of C++.

12.10 Explain how the `switch` statement of C# is safer than that of Java.

12.11 What parameter-passing methods are available in C# that are not available in Java?

12.12 What characteristic is specified by attaching `virtual` to a C# method?

12.13 What does it mean when a C# method includes the `new` modifier?

12.14 Where are C# `struct` objects allocated?

12.15 Explain properties and why they are useful.

12.16 What is a delegate, and what is one common use for delegates?

12.17 What is console input and output?

12.18 What are the two kinds of disadvantages of scripting languages when used for supporting dynamic documents?

12.19 What exactly is a code-behind file?

12.20 From what class does an ASP.NET document class that does not use a code-behind file inherit?

12.21 From what class does an ASP.NET document class that does use a code-behind file inherit?

12.22 What kind of code is placed in a render block?

12.23 What kind of code is placed in a script element?

12.24 Describe what is specified by the `@Page` attribute `Src`.

12.25 What are the two categories of ASP.NET server controls?

12.26 What are the two events that can be raised by the HTML controls?

12.27 Do all of the HTML controls raise events?

12.28 What is the syntactic difference between an XHTML widget and its corresponding HTML control?

12.29 Why do ASP.NET server-side forms not require an `action` attribute?

12.30 What is a postback?

12.31 What is the purpose of the hidden control `ViewState`?

12.32 How can an HTML checkbox control be forced to cause a postback when it is checked?

12.33 What are the four page-level events?

12.34 Explain auto event wireup.

12.35 Explain briefly the two ways to create and register event handlers for HTML controls.

12.36 What is the purpose of the `Xml` control?

12.37 Why should form data validation be done on the server as well as the client?

12.38 What is the difference between an HTML control that includes the `runat` attribute set to `"server"` and one that does not?

Exercises

12.1 Modify the ASP.NET document `ex1.aspx` to get a number between 10 and 100 from the keyboard. The code must verify that the number is in the correct range and then fill the array, which must be defined to have 100 elements, with random numbers and display the values.

12.2 Modify the ASP.NET document `ex2.aspx` and its accompanying code-behind file, `ex2.aspx.cs`, to the specification of Exercise 12.1.

12.3 Modify the ASP.NET document `ex3.aspx` to also use radio buttons to get the marital status of the user (single, married, divorced, widowed) and display the result.

12.4 Modify the ASP.NET document `ex4.aspx` and its accompanying code-behind file, `ex4.aspx.cs`, to add the following: a text box for the user's address and a drop-down list for favorite category of music (rock, rap, country, classical, jazz), which must be populated in the code-behind file. The values of the new controls must be output when a postback is done.

12.5 Modify the ASP.NET document `ex5.aspx` to add the following: a text box for address, which the document must validate to ensure it begins with a number, which is followed by a space and a text string that includes only letters; and a text box to collect a Social Security number, which must be validated to ensure it is in the form ddd-dd-dddd, with no other characters in the text box.

CHAPTER 13

Database Access through the Web

13.1 Relational Databases
13.2 An Introduction to the Structured Query Language
13.3 Architectures for Database Access
13.4 The MySQL Database System
13.5 Database Access with Perl and MySQL
13.6 Database Access with PHP and MySQL
13.7 Database Access with JDBC and MySQL
Summary • Review Questions • Exercises

This chapter begins with brief introductions to relational databases and the Structured Query Language. Then it discusses several different architectures for database access. Next, the primary commands of the MySQL relational database system are introduced. This is followed by three sections, each of which describes a different approach to accessing databases through the Web using MySQL. First, Perl's DBI/DBD architecture is introduced and used to write a CGI program that uses MySQL to access a database. Next, the chapter discusses the use of server-side scripting for building systems for Web access to a database, using PHP as the sample language. Finally, Java's JDBC, which provides database access from Java applications and servlets, is discussed. A complete example of all approaches is provided.

13.1 Relational Databases

A database is a collection of data organized to allow relatively easy access for retrievals, additions, modifications, and deletions. A number of different approaches to structuring data have been developed and used for databases. The most widely used are called *relational database systems*. The original design for relational databases, developed by E. F. Codd in the late 1960s, was based on Codd's mathematical theory of data. A significant number of books have been written to describe the structure and use of relational databases, so it clearly is a large and complex topic. Because just one section of one chapter of this book is devoted to it, that section can provide only a brief overview. However, that is all that is necessary for our discussion of database access through the Web.

A relational database is a collection of tables of data. Each table can have any number of rows and columns of data, and the data itself can have a variety of different forms. The columns of a table are named. Each row usually contains a value for each column. The rows of a table are often referred to as *entities*. The collection of values in a row represents the *attributes* of the entity. Most tables have one column for special data values that uniquely identify the rows of the table. The values in this special column are called the *primary keys* of the table. Mathematically, the entities of a table are elements of a set, so they must be unique. Both data values and primary key values in a table are sometimes called *fields*.

One way to introduce the basic ideas of a relational database is to develop a simple example. Suppose we need a database that stores information about used Corvettes for sale. We could just make a table named `Corvettes` with a column for the primary key of an entity, which could simply be a sequence of numbers. The table could have a column for the body style of the car, one for the year of manufacture, and one for the state where the car is for sale. It would also be useful to include information about the optional equipment of the cars. If six different kinds of equipment were interesting, that would require six more columns in the table.

The use of six columns of the `Corvettes` table for equipment is wasteful of memory. A better design is to use a second table—say, `Equipment`—to store the various kinds of equipment in which we are interested, such as CD players and automatic transmissions. This table could have just two columns: a primary key and the specific equipment. It would need just six rows.

To make this work, we need a way to relate cars to equipment. This need can be met with a cross-reference table, which has just two columns: one with primary keys from the `Corvettes` table and one with primary keys from the `Equipment` table. We could name this table `Corvettes_Equipment`. Each car in the `Corvettes` table could have several rows in `Equipment`, one for each specific option with which the car is equipped. This table does not need, and therefore does not have, a primary key column.

Another way to store the data in less memory is to not store state names in the main table. Instead, we could move the state names to a new table—say, `States`—and have references to it in the `Corvettes` table. A primary key to

the `States` table, which could be just an integer, would require far less space than a typical state name. A logical data model of the database could be as shown in Figure 13.1.

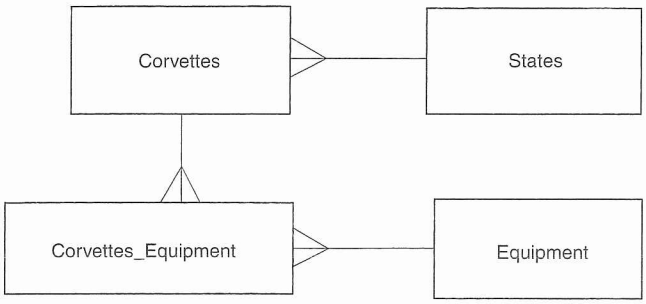


Figure 13.1 The logical data model for a database

The lines between the tables indicate the relationships between the connected tables. For example, the relationship between `Corvettes` and `States` is many to one: There may be many cars for sale in one state, but each car is in just one state. All of the relationships in our model are either one-to-many or many-to-one, depending on your point of view. Note that if we had not used the cross-reference table for this database, the relationship between `Corvettes` and `Equipment` would have been many-to-many.

The implementation of the database is illustrated with short examples of the required tables in Figures 13.2 to 13.5. This database will be used in the remainder of the chapter.

Equip_id	Equip
1	Automatic
2	4-speed
3	5-speed
4	6-speed
5	CD
6	Leather

Figure 13.2 The `Equipment` table

Vette_id	Body_style	Miles	Year	State
1	coupe	18.0	1997	4
2	hatchback	58.0	1996	7
3	convertible	13.5	2001	1
4	hatchback	19.0	1995	2
5	hatchback	25.0	1991	5
6	hardtop	15.0	2000	2
7	coupe	55.0	1979	10
8	convertible	17.0	1999	5
9	hardtop	17.0	2000	5
10	hatchback	50.0	1995	7

Figure 13.3 The `Corvettes` table

State_id	State
1	Alabama
2	Alaska
3	Arizona
4	Arkansas
5	California
6	Colorado
7	Connecticut
8	Delaware
9	Florida
10	Georgia

Figure 13.4 The `States` table

Vette_id	Equip
1	1
1	5
1	6
2	1
2	5
2	6
3	1
3	6
4	2
4	6
5	1
5	6
6	2
7	4
7	6
8	4
8	5
8	6
9	4
9	5
9	6
10	1
10	5

Figure 13.5 The `Corvettes_Equipment` cross-reference table

13.2 An Introduction to the Structured Query Language

The Structured Query Language (SQL) is a standard language for specifying accesses and modifications to relational databases. SQL was originally standardized by the American National Standards Institute (ANSI) and the International

Standards Organization (ISO) in 1986. SQL was significantly expanded and modified in its early years, the result of which was standardized in 1992. This version is often called SQL-2.[1] SQL can be pronounced as either "S-Q-L" or "sequel."

SQL is supported by the databases provided by all major database vendors and is a standard that has truly become *the* standard. It is used to create, query, and modify relational databases, regardless of the particular database vendor.

SQL is quite different from most programming languages; it is actually more like a structured form of English. It was designed to be easily understood and useful for any vendor's database. This section describes some of the basic SQL commands.

The SQL reserved words are not case sensitive, which means that `SELECT`, `select`, and `Select` are equivalent. However, the names of tables and table columns may or may not be case sensitive, depending on the particular database. The whitespace separating reserved words and clauses is ignored, so commands can be spread across several lines if that is more readable. Single quotes (`'`) are used to delimit character strings.

13.2.1 The SELECT SQL Command

`SELECT` clauses are used to specify queries of a database, which is how specific information is requested. The `SELECT` command has three clauses: `SELECT`, `FROM`, and `WHERE`. The general form is as follows:

`SELECT` *column names* `FROM` *table names* `[WHERE` *condition*`];`

The brackets here indicate that the `WHERE` clause is optional.[2] The `SELECT` clause specifies the columns, or attributes, of a table. The `FROM` clause specifies the table or tables to be searched.[3] For example, the following query produces a list of all the values from the `Body_style` column of the `Corvettes` table:

```
SELECT Body_style FROM Corvettes;
```

The `WHERE` clause is used to specify constraints on the rows of the specified tables that are of interest. The following query produces a list of all the values from the `Body_style` column of the `Corvettes` table that have a `Year` column value greater than `1994`:

```
SELECT Body_style FROM Corvettes WHERE Year > 1994;
```

An asterisk (`*`) as the `SELECT` clause value means to select all the columns of the specified table that meet the condition specified in the `WHERE` clause.

---

1. The current version of the SQL standard is SQL-3. It has not yet become widely used.

2. Actually, although the `WHERE` clause is often used, several other clauses can also appear in a `SELECT` command.

3. A `SELECT` command that specifies more than one table produces a join of the tables. Join operations are discussed in Section 13.2.2.